NEREUS 1.0

API Reference

Version 1.0, October 2026

Contents

1 Introduction **3**
- 1.1 Architecture . . . 3
- 1.2 Conventions . . . 3

2 Getting Started with a Custom Method **4**
- 2.1 Build . . . 4
- 2.2 Run a case . . . 4
- 2.3 The case file . . . 5
- 2.4 The program . . . 5
- 2.5 A method of your own . . . 6

3 SimulationManager.h **9**

4 MAC.h **10**
- 4.1 Types . . . 10
- 4.2 Fields . . . 11
- 4.3 Construction . . . 11
- 4.4 Spacing and geometry . . . 12
- 4.5 Field access . . . 14
- 4.6 Cells and faces . . . 15
- 4.7 Boundaries, sources and initial state . . . 17
- 4.8 Differential operators . . . 20
- 4.9 Interpolation and sampling . . . 20
- 4.10 Diagnostics . . . 21
- 4.11 Scalar registry . . . 22

5 Methods, settings and steps (Definitions.h) **23**
- 5.1 The method row . . . 23
- 5.2 Settings tables . . . 25
- 5.3 Steps . . . 26

6 The pressure solver (`Methods/PPESolver/`) **27**

7 Scalar transport **28**

8 Particles **29**
- 8.1 Contact response . . . 30

8.2 Contact on grid methods . . . 30
8.3 Contact on gridless methods . . . 30

9 The Configuration file **31**
9.1 The run . . . 31
9.2 Domain and grid . . . 32
9.3 Fluid and reference values . . . 32
9.4 Time stepping and the end of the run . . . 33
9.5 Output . . . 33
9.6 The pressure block . . . 34
9.7 Particle options . . . 34
9.8 The method blocks . . . 34

10 The scene **36**
10.1 Probes . . . 36

11 Telemetry **37**

12 The user interface **38**

1 Introduction

NEREUS solves the incompressible Navier–Stokes equations in two and three dimensions. It ships three families of methods. ADI is an Eulerian method with alternating-direction implicit momentum sweeps. FLIP is a hybrid particle–grid method. SPH is a Lagrangian particle method. The pressure Poisson system is solved on the CPU with Eigen or AMGCL, or on an NVIDIA GPU with AMGX.

This document describes the classes and functions that a method author or a program uses. It does not describe the internals of the user interface or of `Utils.h`. These are closed boxes.

1.1 Architecture

The run state is global.
`SIMULATION` (`Definitions.h`) holds the configuration, the two grids, the particle set, the telemetry and the export settings. `GRID_ANT` is the field at the start of a step. `GRID_SOL` is the field the step builds. A solver reads the settings it needs from `SIMULATION`. It works only on the grids and particles it is handed.

A method is a registered row.
Each method registers one `SimulationMethod` row. The row gives the method's family, capabilities, settings and initialiser. The library itself names no method. The program registers the methods it wants, and the config's `TYPE` picks one.

A step is a registered function.
The method's initialiser registers its step functions in order. The shared pressure solver registers its own two steps, the pressure solve and the projection. The method only asks for it. `SimulationManager::StepSimulation` runs the steps. Then it does the work that no solver owns. It computes the residual and the telemetry, copies the grid, writes the exports, checks the stopping rule and saves checkpoints.

The grid is `MAC`, a staggered marker-and-cell grid, uniform or stretched.
The velocity lives on the cell faces. The pressure and any registered scalars live at the cell centres. Every cell has a type, and the grid holds the spacing tables and the boundary conditions as recorded index lists.

1.2 Conventions

- **Index order.** Every accessor takes `(i, j, k)`. Here `i` is the row (y), `j` is the column (x) and `k` is the layer (z). The 2D overloads take `(i, j)`. All fields are stored in this order.
- **Staggering.** `u` sits on the x faces, `v` on the y faces and `w` on the z faces. `p` and the scalars sit at the cell centres. A velocity component has one more entry along its own axis, so `u` has $(N_x{+}1) \times N_y \times N_z$ entries and `p` has $N_x \times N_y \times N_z$. Face `j` of `u` lies between cells `j-1` and `j`.
- **Components and axes** are the enums `MAC::Component` (`U`, `V`, `W`, `P`) and `MAC::Axis` (`X`, `Y`, `Z`), written `MAC::U` or `MAC::X`.
- **Cell types.** `FLUID_CELL` (0), `SOLID_CELL` (1), `EMPTY_CELL` (2), `INFLOW_CELL` (3), `PRESSURE_CELL` (4) and `OUTFLOW_CELL` (5). A cell type says what the solvers do with a cell. `SOLID_CELL` is a wall or an obstacle. `EMPTY_CELL` is air in FLIP, and a pressure outlet at zero pressure on the ring. `PRESSURE_CELL` is a pressure outlet with a given pressure. `INFLOW_CELL` is an inlet and `OUTFLOW_CELL` a continuity outflow.
- **The ring.** The grid has one extra layer of cells outside the domain on every side, the *ring*. It carries the boundary conditions. So the walls lie exactly on the faces of the domain, and `GRID_SIZE` counts only the cells inside the domain.
- **Coordinates** are physical, in metres, read from the spacing tables.
- **Units** are SI throughout:

Quantity	Unit	Note
lengths, coordinates, `dh`	m	
time, `dt`, `t`	s	
velocity	m/s	
pressure	Pa	the real pressure, not divided by the density
density `DENSITY`	kg/m^3	
viscosity `VISCOSITY`	m^2/s	kinematic
body force `Fu`, `Fv`, `Fw`, scene `force`	m/s^2	an acceleration, added as $f\,\Delta t$
divergence	1/s	
scalars	any	as the user defines them

2 Getting Started with a Custom Method

This tutorial builds the program, runs a shipped case, and then writes a complete method. The method is `EXPLICIT`, the smallest one in the code: forward-Euler momentum, its own pressure solve by successive over-relaxation (SOR), and the projection. It lives in `src/Methods/Explicit/`, three files of fewer than two hundred lines in all, and adding it changed no file of the library. Every listing is the real code. The functions it calls are described in full in the later sections.

2.1 Build

Build with `make` from the repository's root. It needs a C++17 compiler: GCC on Linux and on Windows (MSYS2), Clang on macOS. Eigen and GLFW ship with the repository. OpenMP is used when the compiler has it. The GPU pressure solver needs AMGX and CUDA, and is built on Linux only, when both are found.

Command	Builds
`make -j`	`bin/nereus`, the program with its window.
`make HEADLESS=1 -j`	`bin/nereus_headless`, with no window and no graphical library linked.
`make NVIDIA_GPU=0 -j`	a program with the CPU pressure solvers only.
`make DEBUG=1 -j`	a debug build, with assertions and bounds checks.
`make info`	nothing. It prints what a build would use.

2.2 Run a case

The folder `configs/` holds one file per case, grouped by benchmark. Each file is complete: the method, the grid, the fluid, the stopping rule, the exports and the scene.

With the window. `bin/nereus` opens the setup screen. Pick a case, for example the cavity `Explicit/ldc_2d_re100_explicit`, and press *Start*. The top bar runs, pauses and steps the simulation.

Without a window. The same file runs from the command line.

```
bin/nereus_headless configs/Explicit/ldc_2d_re100_explicit.cfg
bin/nereus --headless configs/Explicit/ldc_2d_re100_explicit.cfg --progress 100
```

Option	Meaning
`--headless [<config>]`	run the config with no window. The GUI program accepts it too.
`--progress N`	print a progress line every `N` steps (default 100, 0 for none).
`--quiet`	print the summary only.

The exit code is 0 when the stopping rule ended the run, 1 for a usage error, a config file that does not exist or is not valid (below), a `TYPE` that names no registered method, or a config that nothing would stop, 2 when the field stopped being finite (it is checked after every step), and 3 when the run was interrupted. An interrupted run writes its checkpoint first.

The output. Every file of a run goes to one folder,

```
<EXPORT_BASE_PATH>/<config name>/<GRID_SIZE>_<2D|3D>_re<REYNOLDS_NUMBER>/
```

It holds `telemetry.csv`, one row per telemetry interval, and the field frames in `VTK/` as `grid-NNNN.vtr` files for ParaView, numbered by frame (the final field, written when the stopping rule fires, is one more frame). A `.vtr` is a rectilinear grid with one point per cell centre, the ring included. It carries `Velocity` (each component the mean of the cell's two faces), `Pressure`, `Divergence`, `Solid` (the cell type) and one array per registered scalar. The particle methods add particle files and, in 3D, liquid surfaces.

2.3 The case file

A config file has `KEY = value` lines, then blocks for the pressure solver, for each method's settings and for the scene. A key that is left out takes its default (`DIMENSION` defaults to 3, so a 2D case must say so). The reader refuses a file it cannot trust, listing every problem before the run starts: a key no reader knows, at the top level or inside a method block; a number or an integer that does not parse in full (`VISCOSITY = 0.01abc`); a choice outside its list (`CONVECTION_SCHEME = CENTRALL`); a malformed refinement interval; and a block for a method the program did not register. Section 9 lists every key, and Section 11 every telemetry column. This is the lid-driven cavity at Re 100 the tutorial runs, `configs/Explicit/ldc_2d_re100_explicit.cfg`, shortened.

```
DIMENSION = 2
TYPE = EXPLICIT                 # the key of a registered method
THREAD_COUNT = 1
DOMAIN_X0 = 0
DOMAIN_XF = 1
DOMAIN_Y0 = 0
DOMAIN_YF = 1
GRID_SIZE = 32                  # cells across the shorter side
VISCOSITY = 0.01                # kinematic, m^2/s
DENSITY = 1
TIME_STEP = 0.01                # the largest step; ADAPTATIVE_TIMESTEP only shrinks it
ADAPTATIVE_TIMESTEP = 1
MAX_CFL = 0.5
STOP_CONDITION = TIME           # NEVER, TIME or RESIDUAL
STOP_TIME = 60
EXPORT_ENABLED = 1
EXPORT_GRID = 1
EXPLICIT_BEGIN                  # the method's own block, read through its settings table
EXPLICIT_RELAXATION = 1.7
EXPLICIT_TOLERANCE = 1e-06
EXPLICIT_MAX_ITERATIONS = 10000
EXPLICIT_END
SCENE_BEGIN
boundary xmin noslip
boundary xmax noslip
boundary ymin noslip
boundary ymax moving u="1"   # last, so the lid wins the two top corners
SCENE_END
```

The scene gives each face of the domain a boundary kind, here three no-slip walls and a moving lid. Regions add solids or, for a particle method, the initial liquid. Probes add telemetry columns (Section 10). To try another method on the same case, change `TYPE`: `TYPE = ADI` runs the same cavity with the ADI method. The setup screen edits the same file and saves the whole of it back, so a file written by hand and one written on screen are interchangeable.

2.4 The program

A program registers the methods it can run, starts a run from a config file, steps it, and releases the pressure solver at the end. The library names no method, so a program that leaves a method out is built without it.

```
#include "headers/Core/SimulationManager.h"
#include "Methods/ADI/ADISettings.h"
#include "Methods/Explicit/ExplicitSettings.h"

int main()
{
    RegisterADI();                                  // before any config is read
```

```
    RegisterExplicit();
    SimulationManager::InitializeSimulation("configs/Explicit/ldc_2d_re100_explicit.cfg");
    if (SIMULATION.exports.enabled) Exports::InitializeTelemetry();
    SIMULATION.running = true;
    while (SIMULATION.running)                      // false once the stopping rule fires
        SimulationManager::StepSimulation();
    SimulationManager::DisposeSimulation();         // frees the method, the grids and the steps
    return 0;
}
```

The registration comes first because the config's `TYPE` is looked up in the registry, and building the grid asks the method whether it allows a stretched grid. `Exports::InitializeTelemetry` names the telemetry file and writes its header; the file opens at the first row. A program without a window calls it only when exports are on, the window's loop calls it always, since export can be switched on during the run. With a window, the four calls of Section 12 wrap the same loop. The shipped `src/main.cpp` is this program, with both.

2.5 A method of your own

The method is a folder of its own. The Makefile compiles every `src/Methods/*/*.cpp`, so the folder needs no build change, and the program registers the method with one line, `RegisterExplicit();`, once it includes the method's settings header. Every object file is built into one folder and every method is linked into one program, so a `.cpp` file name must be unique across `src/`, and the method's classes, namespaces and global names must not clash with another method's (the explicit method keeps its own inside `Explicit2D`).

```
src/Methods/Explicit/
  Explicit2D.h          the declarations of the step functions
  Explicit2D.cpp        momentum, pressure, projection, stable time step
  ExplicitSettings.h    the settings struct and table, InitializeExplicit, RegisterExplicit
```

Step 1: the settings

The settings are a plain struct with one global instance, and a table (Section 5.2). From the table alone the config reader fills the struct from the `EXPLICIT` block, and the setup screen draws an input box for each real number and a slider for the bounded integer.

```
struct ExplicitSettings
{
    double relaxation = 1.7;      // SOR factor: 1 is plain Gauss-Seidel
    double tolerance = 1e-6;      // stop when no cell's equation is off by more than this times the largest
     source
    int maxIterations = 10000;
};
inline ExplicitSettings EXPLICIT_SETTINGS;

inline const SettingField EXPLICIT_FIELDS[] = {
    DoubleSetting("EXPLICIT_RELAXATION", "SOR relaxation", "SOR over-relaxation factor, 0.5 to 2",
                  offsetof(ExplicitSettings, relaxation), 1.7, 0.5, 2.0),
    DoubleSetting("EXPLICIT_TOLERANCE", "Pressure tolerance", "Largest residual relative to the largest right-
     hand side",
                  offsetof(ExplicitSettings, tolerance), 1e-6, 1e-14, 1.0),
    IntSetting("EXPLICIT_MAX_ITERATIONS", "Max sweeps", "", offsetof(ExplicitSettings, maxIterations), 10000,
     1, 1e7),
};
```

Step 2: the row

The row is everything the rest of the code knows about the method (Section 5).

```
static inline void RegisterExplicit()
{
    RegisterSimulationMethod({"EXPLICIT", "Explicit (Eulerian)",
        "Forward Euler momentum and an SOR pressure solve, 2D uniform grids",
        MethodFamily::EULERIAN, true, false, CAP_GRID | CAP_BOUNDARY_CELLS | CAP_FIXED_REGION,
        InitializeExplicit, nullptr,
        &EXPLICIT_SETTINGS, EXPLICIT_FIELDS, (int)(sizeof(EXPLICIT_FIELDS) / sizeof(EXPLICIT_FIELDS[0])),
     nullptr,
        nullptr, Explicit2D::StableTimestep, nullptr, nullptr, nullptr});
```

```
}
```

The row runs in 2D only. It leaves three capabilities off on purpose: without `CAP_RECTILINEAR` it is given a uniform grid, without `CAP_SCALARS` no scalars are transported, and without `CAP_PRESSURE_SOLVER` the setup screen does not show the shared pressure solver's settings. `stableTimestep` gives the method's own stability limit (Step 6). The other questions are null, so the manager answers them from the grid.

Step 3: the steps

The initialiser seeds the grids from the scene and registers three step functions (Section 5.3). The manager runs them in order every time step and times each one.

```
static inline void InitializeExplicit()
{
    MAC::InitializeSimulationGrids();
    RegisterSimulationStep(Explicit2D::Momentum, "momentum", 0);
    RegisterSimulationStep(Explicit2D::Pressure, "pressure", 4);
    RegisterSimulationStep(Explicit2D::Projection, "projection", 6);
}
```

`MAC::InitializeSimulationGrids` (Section 4.3) seeds both of the run's grids from the scene and gives each the boundary functions.

Step 4: momentum

On every face between two fluid cells,

$$\mathbf{u}^* = \mathbf{u}^n + \Delta t \left(-(\mathbf{u}^n \cdot \nabla)\mathbf{u}^n + \nu \nabla^2 \mathbf{u}^n + \mathbf{f} \right),$$

with central differences. The Laplacian of a face value u sums over its four neighbours u_m,

$$\nabla^2 u \approx \frac{1}{h^2} \sum_m f_m \, (u_m - u),$$

where f_m is the wall factor of Section 4.7. It is the ratio between the distance the stencil assumes, one cell, and the true distance to where the neighbour's value belongs.

- **A face in the middle of the domain.** Its neighbour u_m is another face one cell away, where its value is stored. The distance is right, so $f_m = 1$.
- **A face in the first row beside a no-slip wall.** The neighbour across the wall is the ring value, which holds the wall velocity u_w. That value belongs on the wall, the boundary face, which is only half a cell away. The difference $u_w - u$ must be divided by $h/2$, not h, so $f_m = 2$: the term $2(u_w - u)$ is the same as a ghost value $2u_w - u$ reflected through the wall, one cell away.

A slip wall, an inlet or an outlet has its value where the stencil expects it, so $f_m = 1$ there too. The function `MAC::WallLinkFactor` returns f_m for every coupling, so the loop needs no special case for walls. The convection is written in the advective form, $(\mathbf{u} \cdot \nabla)\mathbf{u}$, which keeps a uniform field uniform but does not conserve momentum or kinetic energy discretely. Its central difference next to a wall reads the ring value as if it were a full cell away; this simple method does not correct that, which leaves the convective term first order in the first row beside a wall.

```
static double Laplacian(MAC *g, MAC::Component c, int i, int j)
{
    const double u = g->GetComponent(c, i, j);
    double sum = 0.0;
    for (int s = -1; s <= 1; s += 2)
        sum += g->WallLinkFactor(c, MAC::X, i, j, s) * (g->GetComponent(c, i, j + s) - u)
             + g->WallLinkFactor(c, MAC::Y, i, j, s) * (g->GetComponent(c, i + s, j) - u);
    return sum / (g->dh * g->dh);
}

void Explicit2D::Momentum(MAC *ant, MAC *sol, ParticleSet *, double time)
{
    const double dt = SIMULATION.dt, nu = SIMULATION.EPS, h2 = 2.0 * ant->dh;
    sol->CopyFlowFields(*ant);
    for (int i = 1; i < ant->Ny - 1; i++)
        for (int j = 1; j < ant->Nx; j++)
```

```
            if (ant->FaceGate(MAC::U, i, j) == GateState::OPEN)
            {
                const double u = ant->GetU(i, j), v = ant->getVatU(i, j);
                const double dudx = (ant->GetU(i, j + 1) - ant->GetU(i, j - 1)) / h2;
                const double dudy = (ant->GetU(i + 1, j) - ant->GetU(i - 1, j)) / h2;
                sol->SetU(i, j, u + dt * (-u * dudx - v * dudy + nu * Laplacian(ant, MAC::U, i, j)));
            }
    /* ... the same loop for v ... */
    sol->AddForce(time, dt);
    sol->EnforceVelocityBC(time + dt);
}
```

The step leaves the faces next to the ring alone. `EnforceVelocityBC` sets them for every boundary kind, whether wall, inlet or outlet, so the method runs any scene the code can describe, obstacles included.

Step 5: pressure and projection

The pressure solves

$$\nabla^2 p = \frac{\rho}{\Delta t}\,\nabla \cdot \mathbf{u}^*$$

on the fluid cells. For a cell with pressure p and right-hand side b, scaled by h^2, the discrete equation is

$$\sum_n c_n\,(p_n - p) = b, \qquad c_n = \begin{cases} 1 & \text{neighbour fluid (an unknown),} \\ 2 & \text{neighbour a pressure-outlet ring cell (a known value half a cell away),} \\ 0 & \text{neighbour a wall or an inlet (known velocity: no flux).} \end{cases}$$

With $D = \sum_n c_n$, the residual of the cell and the SOR update are

$$r = \sum_n c_n\,p_n - D\,p - b, \qquad p \leftarrow p + \omega\,\frac{r}{D}.$$

For $\omega = 1$ this is Gauss–Seidel: r/D is the change that satisfies the cell's equation. The relaxation ω is `EXPLICIT_RELAXATION`.

```
double sum = 0.0, diag = 0.0;
for (int n = 0; n < 4; n++)
{
    const int ni = i + di[n], nj = j + dj[n];
    const bool outlet = g->IsPressureOutlet(ni, nj);
    const double c = g->IsFluid(ni, nj) ? 1.0 : (outlet ? 2.0 : 0.0);
    sum += c * g->GetP(ni, nj);
    diag += c;
}
const double p = g->GetP(i, j);
const double r = sum - diag * p - b[i * nx + j];   // how far the equation is from being satisfied
g->SetP(i, j, p + w * r / diag);                   // r / diag zeroes it; w over-relaxes
```

`IsFluid` and `IsPressureOutlet` are the grid's cell queries (Section 4); the second is true for the two ring types of a pressure outlet, `EMPTY_CELL` and `PRESSURE_CELL`. The sweeps stop when the largest $|r|$ is below `EXPLICIT_TOLERANCE` times the largest $|b|$, or after `EXPLICIT_MAX_ITERATIONS` sweeps. A method that brings its own solver also decides what to do and report when it does not converge; the explicit method stops silently. With no outlet the pressure is fixed only up to a constant, and its mean is set to zero afterwards. `EnforcePressureBC` runs before the sweeps, for the outlet values, and after them, so each wall ring cell copies its neighbour.

The projection then corrects every face that is not a wall,

$$\mathbf{u}^{n+1} = \mathbf{u}^* - \frac{\Delta t}{\rho}\,\nabla p,$$

with the gradient taken over the half cell on a face beside an outlet ring cell.

```
if (g->FaceGate(MAC::U, i, j) != GateState::WALL)
{
    const bool half = g->IsPressureOutlet(i, j) || g->IsPressureOutlet(i, j - 1);
    g->SetU(i, j, g->GetU(i, j) - (half ? 2.0 : 1.0) * k * (g->GetP(i, j) - g->GetP(i, j - 1)));
}
```

Step 6: the stable time step

Forward Euler with central differences, in 2D, is stable when the step is below the two limits

$$\Delta t \leq \frac{h^2}{4\nu}, \qquad \Delta t \leq \frac{2\nu}{|u|^2},$$

the diffusive limit and the convective one of a central scheme. Together they imply the CFL condition $\Delta t \leq h/|u|$, which the code also checks, as a guard on how far the flow moves in one step. The row's `stableTimestep` returns the smallest of the three times `MAX_CFL`. The manager uses it under `ADAPTATIVE_TIMESTEP`. Without that key `TIME_STEP` is used as given, and nothing checks it.

```
double Explicit2D::StableTimestep()
{
    const double h = SIMULATION.GRID_ANT->dh, nu = SIMULATION.EPS;
    const double u = std::max(SIMULATION.GRID_ANT->GetMaxVelocity(), 1e-6);
    return MAX_CFL * std::min({h / u, h * h / (4.0 * nu), 2.0 * nu / (u * u)});
}
```

Step 7: run it

Build, then run the case of the tutorial.

```
make HEADLESS=1 -j
bin/nereus_headless configs/Explicit/ldc_2d_re100_explicit.cfg
```

Every step the manager runs `momentum`, `pressure` and `projection`, then the residual, the telemetry and the exports (Section 3). At $t = 60$ the stopping rule fires and writes the final field to `VTK/`. The telemetry has one `<name>_timing` column per step, so the method is profiled with no code of its own.

Possibilities for extension

- Add an obstacle: `region solid cylinder axis=z center=(0.5,0.5) radius=0.1` in the scene. The method needs no change, since `FaceGate` and `EnforceVelocityBC` handle the solid.
- Report the SOR sweep count: pass a counter function as the fourth argument of `RegisterSimulationStep`, and the telemetry gains a `pressure_counter` column.
- Use the shared pressure solver instead of SOR: call `PressureSolver2D::InitializePressureSolver` in the initialiser, drop the two pressure steps, and set `CAP_PRESSURE_SOLVER` on the row (Section 6).

3 SimulationManager.h

`src/headers/Core/SimulationManager.h` is header-only. The class builds a run from a config file and advances it one step at a time. It names no method, holds no data and draws nothing. All the state lives on `SIMULATION`, and the program registers the methods first (Section 5). The three functions below are all a program needs: the class's other members are the internal parts of a step.

```
static void InitializeSimulation(const std::string &configFile = "");
```

Builds a run from a config file. It reads the config, which builds the grids and the scene, and sets the number of OpenMP threads. It clears the step list, the particle set and the telemetry. Then it calls the active method's initialiser, which registers the steps. It stops with an error when no method is registered.

Parameters

- `configFile` – the path to a `.cfg` file. An empty string means `simulation_config.txt` in the working directory.

```
static void StepSimulation();
```

Advances the run by one time step, when `SIMULATION.running` or `SIMULATION.stepOnce` is set. While the run is paused it changes nothing, so a window can keep drawing. One step does the following.

1. It performs a pending checkpoint load or save, and computes a new `dt` under `ADAPTATIVE_TIMESTEP`.

2. It shortens `dt` if the step would pass the stop time, and sets `SIMULATION.time` to the time the step solves for, $t + \Delta t$.
3. It zeroes `GRID_SOL` and runs every registered step in order, timing each one.
4. It computes the residual, the largest change between the two grids, and the telemetry, the divergence included.
5. It copies `GRID_SOL` into `GRID_ANT`, and advances the clock and the step count.
6. It writes the exports that are due, checks the stopping rule and saves a checkpoint when one is due. When the rule fires it sets `SIMULATION.running` to false.

```
static void DisposeSimulation();
```

Ends a run. It calls the active method's `dispose`, closes the telemetry file, deletes the two grids, clears the step list and returns `SIMULATION` to its defaults. `InitializeSimulation` can then start another run in the same process, and that run is the one a fresh process would make, to the last bit. The viewer's Restart button uses it.

Example

```
RegisterADI();
SimulationManager::InitializeSimulation("configs/Ghia/2d_ghia_re100.cfg");
SIMULATION.running = true;
while (SIMULATION.running)
    SimulationManager::StepSimulation();
SimulationManager::DisposeSimulation();
```

4 MAC.h

`src/headers/Core/MAC.h` and `src/MAC.cpp`. The staggered marker-and-cell grid. Short member functions are defined in the header. Longer ones are in `MAC.cpp`. Almost every function has a 2D form that takes `(i, j)` in place of `(i, j, k)`. Where both exist they are described together.

4.1 Types

```
enum Component { U = 0, V = 1, W = 2, P = 3 };
enum Axis { X = 0, Y = 1, Z = 2 };
```

The staggered fields and the axes. They are plain enums inside the class, so a value can index a table directly. A velocity component and its own normal axis have the same number.

```
static Axis AxisOf(Component c);
```

The normal axis of a velocity component.

Parameters

- `c` – `MAC::U`, `V` or `W`. `P` has no normal axis and is not a valid argument: the result is not one of the three axes.

Returns `X` for `U`, `Y` for `V` and `Z` for `W`.

```
static Component ComponentAlong(Axis a);
```

The velocity component along an axis, the inverse of `AxisOf`.

Parameters

- `a` – `MAC::X`, `Y` or `Z`.

Returns `U` for `X`, `V` for `Y` and `W` for `Z`.

Example

```
double h = g.Width(MAC::AxisOf(c), m);   // the width of cell m along the axis component c is normal to
```

4.2 Fields

Field	Type	Meaning
`Nx, Ny, Nz`	`int`	cells along x, y and z, with the ring.
`omega`	`Domain`	the physical extents of the domain.
`dh`	`double`	the grid spacing. On a stretched grid it is the *smallest* cell.
`is2D`	`bool`	true for a 2D grid.
`nonUniform`	`bool`	true when the spacing tables are stretched.

4.3 Construction

A method does not build its grids. The config reader builds the run's two grids, `GRID_ANT` and `GRID_SOL`, before any method code runs, and the method's initialiser seeds them with `InitializeSimulationGrids`. A grid the method keeps for itself is made with `InitializeFromGrid`. The constructor and `InitializeGrid`, at the end of this section, are for programs and tests that build a grid from nothing.

```
static void InitializeSimulationGrids();
```

The start every grid method shares, called from the method's `initialize`. It calls `InitializeFromScene` at time 0 on both of the run's grids, `GRID_ANT` and `GRID_SOL`, so the first step starts from a consistent field.

Example

```
static inline void InitializeMyMethod()
{
    MAC::InitializeSimulationGrids();
    RegisterSimulationStep(MyMethod::Momentum, "momentum", 0);
}
```

```
void InitializeFromScene(double t = 0.0);
```

Seeds this grid from the scene. It applies the scene's `initial` field, gives the grid the scene's boundary functions, and applies the boundary at time `t`. The grid must already be sized.

Parameters

- `t` – the time the boundary is evaluated at, for a time-dependent boundary.

```
void InitializeFromGrid(MAC &model, double t = 0.0);
```

Builds this grid as a copy of another, for the grids a method keeps for itself (the ADI sweeps use it for theirs). It sizes the grid like `model`, copies its cell types and its velocity and pressure, takes its boundary functions, and applies the boundary at time `t`. Registered scalars are not copied.

Parameters

- `model` – the grid to copy, usually one of the run's grids. Its boundary functions must be set.
- `t` – the time the boundary is evaluated at.

Example

```
static MAC work;                                  // a method's own grid
work.InitializeFromGrid(*SIMULATION.GRID_ANT, SIMULATION.time);
```

How the grid is built

The base spacing. `GRID_SIZE` is the number of cells across the shorter of the domain's x and y lengths, so the base spacing is $h = \min(L_x, L_y)/\texttt{GRID_SIZE}$; the z length does not enter it. Every axis gets $n = \lfloor L/h \rfloor$ cells of width h. When L is not a whole number of cells the last face falls up to one cell short of the domain's far edge, and the boundary lies there.

Refinement. With `NON_UNIFORM_GRID = 1`, a method with `CAP_RECTILINEAR` and at least one interval in `REFINE_X`, `REFINE_Y` or `REFINE_Z`, each axis is built from the intervals on it. An interval asks for a local factor $F(x)$: cells of size $h/F(x)$. Where intervals overlap, the largest factor wins.

- `hard`: F is the interval's factor throughout. The interval's edges are kept exactly, and it holds a whole number of equal cells, round($L_{\text{band}}F/h$), so a band around a body stays where it is put.
- `ramp`: F grows linearly from 1 at `start` to the factor at `end`, so the cell size h/F falls hyperbolically, quickly near `start` and slowly near `end`. A large factor over a short ramp therefore gives a large jump in size at its start; several nested ramps ending at the same edge follow a smoother law. `ramp_back` is the mirror.
- `graded`: F rises linearly from 1 at both ends to the factor in the middle.

Between the edges of the `hard` intervals, the grid marches cell by cell with size $h/F(x)$, then scales that span's cells so they fill it exactly. So no short leftover cell appears, and the cell-size ratios are kept. An axis with only `hard` intervals splits each base cell whose centre lies in an interval into F equal cells. The number of cells is known only after this construction: `Nx`, `Ny` and `Nz` hold it, and the console reports it.

The ring. One ring cell is added outside each end of every axis, as wide as the interior cell beside it. So the walls sit on the domain's faces, and the geometry about a wall is symmetric on any grid.

What a stretched grid changes. `dh` becomes the smallest cell, which is what the adaptive time step uses. `nonUniform` is set whenever the config asks for refinement and the method allows it, even when no interval is active and the tables come out uniform. A method without `CAP_RECTILINEAR` is given the uniform grid, with a warning. On a uniform grid every table entry is the same double the plain arithmetic $x_0 + jh$ gives, so uniform results do not depend on the tables.

A grid can also be built from nothing, as the config reader and the tests do:

```
MAC();
```

Creates an empty grid. Call `InitializeGrid` before using it.

```
void InitializeGrid(Domain omega, bool is2d = false);
void InitializeGrid(Domain2D omega);
```

Sizes the grid from `SIMULATION.GRID_SIZE` and builds its spacing tables, as described above. It allocates `u`, `v`, `w` and `p` as zeros, makes every cell a `FLUID_CELL` and resizes the registered scalars.

Parameters

- `omega` – the domain.
- `is2d` – builds the 2D grid. This is the same as the `Domain2D` form.

```
void SetSolidMask(int (*SolidMaskFunction)(int, int, int));
void SetSolidMask(int (*SolidMaskFunction)(int, int));
```

Sets the type of every cell from a function of its indices. A run gets the function from the scene.

Parameters

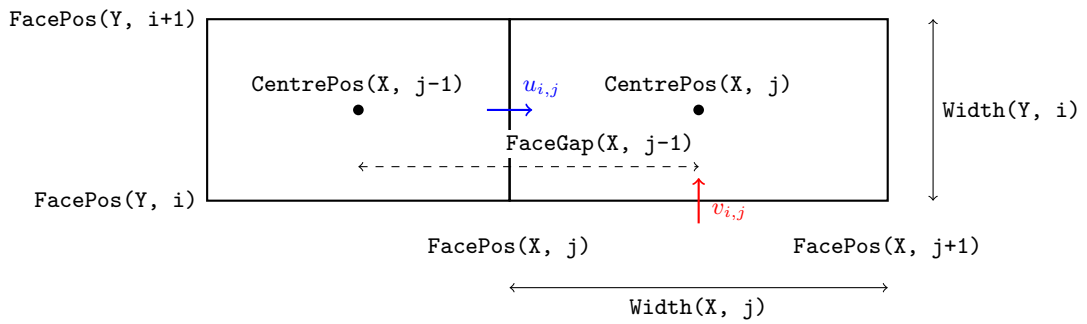
- `SolidMaskFunction` – returns a cell type for `(i, j[, k])`.

Example

```
MAC g;
g.InitializeGrid(SIMULATION.domain);                // 3D. Sizes, tables, zeroed fields
g.SetSolidMask(SIMULATION.SolidMaskFunction);       // cell types from the scene
```

4.4 Spacing and geometry

Each axis has four tables, indexed from 0. They hold the cell widths, the face positions (N+1 entries), the centre positions, and the gaps. A gap is the distance between the centres of cells n and n+1. All lookups take constant time. One set of functions reads them. The axis `a` comes first. `MAC::X` goes with index j, `MAC::Y` with i and `MAC::Z` with k. On a uniform grid every width and every gap is exactly `dh`.



The figure shows two neighbouring cells $(i, j-1)$ and (i, j) of one row in 2D. $u_{i,j}$ sits at (`FacePos(X, j)`, `CentrePos(Y, i)`). $v_{i,j}$ sits at (`CentrePos(X, j)`, `FacePos(Y, i)`). p sits at the centre. `FaceGap(X, j-1)` is the mean of the widths of cells `j-1` and `j`. It is the distance d_f of face j in the pressure coefficient A_f/d_f.

```
double Width(Axis a, int n) const;
```

The width of a cell along an axis.

Parameters

- `a` – the axis, `MAC::X`, `Y` or `Z`.
- `n` – the cell index along that axis (`j`, `i` or `k`).

Returns the width, in metres.

```
double FacePos(Axis a, int n) const;
```

The position of a face, the low face of cell `n`.

Parameters

- `a` – the axis.
- `n` – the face index along that axis, from 0 to the number of cells.

Returns the coordinate, in metres.

```
double CentrePos(Axis a, int n) const;
```

The position of the centre of a cell.

Parameters

- `a` – the axis.
- `n` – the cell index along that axis.

Returns the coordinate, in metres.

```
double FaceGap(Axis a, int n) const;
```

The distance between the centres of cells `n` and `n+1`, the d_f of the face between them.

Parameters

- `a` – the axis.
- `n` – the cell index along that axis.

Returns the distance, in metres.

```
const std::vector<double> &Faces(Axis a) const;
const std::vector<double> &Centres(Axis a) const;
```

The whole face table or centre table of an axis, for code that searches or plots it.

Parameters

- `a` – the axis.

Returns a read-only reference to the table.

```
double CellVolume(int i, int j, int k) const;
```

The volume of a cell. In 2D it is an area.

Parameters

- `i, j, k` – the cell. The 2D form drops `k`.

Returns the volume, in cubic metres (square metres in 2D).

```
double FaceArea(Axis a, int i, int j, int k) const;
```

The area of a cell's faces normal to an axis. In 2D it is a length.

Parameters

- `a` – the axis the faces are normal to.
- `i, j, k` – the cell. The 2D form drops `k`.

Returns the area, in square metres (metres in 2D).

Example

```
// Net volume flux out of cell (i, j, k), correct on a stretched grid
double flux = (g.GetU(i, j + 1, k) - g.GetU(i, j, k)) * g.FaceArea(MAC::X, i, j, k)
            + (g.GetV(i + 1, j, k) - g.GetV(i, j, k)) * g.FaceArea(MAC::Y, i, j, k)
            + (g.GetW(i, j, k + 1) - g.GetW(i, j, k)) * g.FaceArea(MAC::Z, i, j, k);
```

4.5 Field access

```
double GetU(int i, int j, int k);
double GetV(int i, int j, int k);
double GetW(int i, int j, int k);
double GetP(int i, int j, int k);
void SetU(int i, int j, int k, double value);
void SetV(int i, int j, int k, double value);
void SetW(int i, int j, int k, double value);
void SetP(int i, int j, int k, double value);
```

Read or write one stored value of a field. The 2D forms drop `k` and have no `W`.

Parameters

- `i, j, k` – the face (velocity) or the cell (pressure). See the staggering convention.
- `value` – the value to store.

Returns the stored value (`Get`).

```
double GetComponent(Component c, int i, int j, int k);
void SetComponent(Component c, int i, int j, int k, double value);
```

The same, with the field chosen at run time.

Parameters

- `c` – `MAC::U`, `V`, `W` or `P`.
- `i, j, k` – the face or the cell. The 2D form drops `k`.
- `value` – the value to store.

Returns the stored value (`GetComponent`).

Example

```
for (MAC::Component c : {MAC::U, MAC::V})
    g.SetComponent(c, i, j, 0.0);
```

```
VectorXd &Field(Component c);
const VectorXd &Field(Component c) const;
```

The whole array of a field, for work on all values at once, such as copies and norms.

Parameters

- `c` – `MAC::U`, `V`, `W` or `P`.

Returns a reference to the array.

```
void ClearFlowFields();
```

Sets the velocity and the pressure to zero.

```
void CopyFlowFields(MAC &grid);
void CopyScalarFields(MAC &grid);
void CopyGrid(MAC &grid);
```

Copy another grid of the same size into this one. `CopyFlowFields` copies the velocity, the pressure and the cell types. `CopyScalarFields` copies the registered scalars, matched by name. `CopyGrid` does both.

Parameters

- `grid` – the grid to copy from.

4.6 Cells and faces

```
int GetSolid(int i, int j, int k) const;
void SetSolid(int i, int j, int k, int value);
```

Read or write the type of a cell. Changing whether a cell is solid, or changing a ring cell, makes the grid record its boundary lists again on their next use.

Parameters

- `i, j, k` – the cell. The 2D form drops `k`.
- `value` – a cell type, `FLUID_CELL` to `OUTFLOW_CELL`.

Returns the cell type (`GetSolid`).

```
enum class GateState { OPEN, WALL, FREE };
```

How a cell, or a face, connects the two control volumes on either side of it. The two functions below return it. `OPEN` is fluid to fluid. `WALL` is through a solid. `FREE` is through air, an inlet or a pressure cell.

```
GateState CellGate(int i, int j, int k) const;
```

How a cell connects its neighbours.

Parameters

- `i, j, k` – the cell. The 2D form drops `k`.

Returns `WALL` for a solid cell, `OPEN` for a fluid cell, `FREE` otherwise.

```
GateState FaceGate(Component c, int i, int j, int k) const;
```

How a face connects the two cells on either side of it.

Parameters

- `c` – the velocity component the face belongs to.
- `i, j, k` – the face. The 2D form drops `k`.

Returns `WALL` if either cell is solid, `OPEN` if both are fluid, `FREE` otherwise.

Example

```
if (g.FaceGate(MAC::U, i, j) == GateState::OPEN)   // both cells fluid: solve this face
    ...
```

```
bool IsFluidFace(Component c, int i, int j, int k) const;
```

Whether the flow is solved on a face.

Parameters

- `c` – the velocity component the face belongs to.
- `i, j, k` – the face. The 2D form drops `k`.

Returns true when the face touches no solid and at least one fluid cell.

```
bool IsInteriorFluid(int i, int j, int k) const;
```

Whether a cell is fluid and inside the domain.

Parameters

- `i, j, k` – the cell. The 2D form drops `k`.

Returns true for a fluid cell that is not on the ring.

```
bool IsCellType(int i, int j, int k, int type) const;
```

Whether a cell has a given type.

Parameters

- `i, j, k` – the cell. The 2D form drops `k`.
- `type` – a cell type, `FLUID_CELL` to `OUTFLOW_CELL`.

Returns true when `GetSolid(i, j, k) == type`.

```
int BoundaryKindAt(int i, int j, int k) const;
```

The boundary kind of a cell. The three walls share `SOLID_CELL`, so the cell type alone cannot tell a no-slip, a slip and a moving wall apart; the kind can. A ring cell takes the kind of the face that owns it (the scene's, Section 4.7); an interior solid is a no-slip wall.

Parameters

- `i, j, k` – the cell. The 2D form drops `k`.

Returns a `BoundaryKind` (`BC_NOSLIP` to `BC_OUTFLOW`), or -1 for an interior cell that is not solid.

```
bool IsFluid(int i, int j, int k) const;           // FLUID_CELL
bool IsSolidWall(int i, int j, int k) const;       // SOLID_CELL: any wall or obstacle
bool IsNoSlipWall(int i, int j, int k) const;      // kind BC_NOSLIP (and every obstacle)
bool IsSlipWall(int i, int j, int k) const;        // kind BC_SLIP
bool IsMovingWall(int i, int j, int k) const;      // kind BC_MOVING
bool IsInlet(int i, int j, int k) const;           // INFLOW_CELL
bool IsOutflow(int i, int j, int k) const;         // OUTFLOW_CELL
bool IsPressureOutlet(int i, int j, int k) const;  // EMPTY_CELL or PRESSURE_CELL
```

Shorthands for the common questions, built on `IsCellType` and `BoundaryKindAt`. `IsPressureOutlet` is true for a cell whose pressure is known: a pressure-outlet ring cell, at $p = 0$ (`EMPTY_CELL`) or at a prescribed value (`PRESSURE_CELL`). Under a particle method an air cell is also `EMPTY_CELL`, at the free-surface value $p = 0$, so it answers true as well.

Parameters

- `i, j, k` – the cell. The 2D forms drop `k`.

Returns true when the cell is of that kind.

Example

```
// the coefficient of a neighbour in a pressure stencil
const double c = g->IsFluid(ni, nj) ? 1.0 : (g->IsPressureOutlet(ni, nj) ? 2.0 : 0.0);
```

```
void ResetFluidCells();
```

Sets every cell that is not a solid, an inlet, an outflow or a pressure cell to `EMPTY_CELL`. A particle method calls it before it marks the cells that hold particles.

4.7 Boundaries, sources and initial state

Each face of the domain has a boundary kind (`BoundaryKind` in `Definitions.h`): `noslip`, `slip`, `moving`, `inlet`, `pressure_outlet` or `outflow`. The grid imposes a condition by writing values into the ring of cells outside the domain, either a prescribed value or a copy of the value inside (zero gradient). Each solver then reads the ring as it needs. A ring cell on an edge or a corner lies on two or three faces. It takes the kind of the face the scene declares last.

The particle methods (FLIP and SPH) support closed domains with fixed walls only: they do not inject particles at an inlet or remove them at an outlet, and their contact treats every wall as stationary. The setup screen warns when a particle method is given an `inlet`, `outflow`, `pressure_outlet` or `moving` face.

Kind	Ring cell	Normal velocity	Tangential velocity	Pressure
`noslip`	`SOLID_CELL`	$u_n = 0$	$u_t = 0$	copied from inside
`moving`	`SOLID_CELL`	$u_n = 0$	$u_t = U_w(\mathbf{x}, t)$	copied from inside
`slip`	`SOLID_CELL`	$u_n = 0$	copied from inside	copied from inside
`inlet`	`INFLOW_CELL`	$u_n = g_n(\mathbf{x}, t)$	$u_t = g_t(\mathbf{x}, t)$	copied from inside
`pressure_outlet`	`EMPTY_CELL`; `PRESSURE_CELL` with `p=`	copied, then set by the projection	copied from inside	$p_b(\mathbf{x}, t)$ on the boundary face, 0 without `p=`
`outflow`	`OUTFLOW_CELL`	copied, then shifted to match the inlets	copied from inside	copied from inside

```
void SetBoundaryFunctions(Vec3 (*velocity)(double, double, double, double),
                          double (*pressure)(double, double, double, double));
void SetBoundaryFunctions(Vec2 (*velocity)(double, double, double),
                          double (*pressure)(double, double, double));
```

Gives the grid the functions the prescribed boundary values come from. A run gets them from the scene.

Parameters

- `velocity` – the velocity at `(x, y[, z], t)`.
- `pressure` – the pressure at `(x, y[, z], t)`. It is used on a pressure outlet.

```
void EnforceVelocityBC(double t);
void EnforcePressureBC(double t);
void EnforceBoundaryConditions(double t);
```

Apply the boundary conditions at time `t`. `EnforceVelocityBC` writes the velocity of the ring and of the boundary faces (below). `EnforcePressureBC` writes the ring pressure. `EnforceBoundaryConditions` calls both. A momentum step calls `EnforceVelocityBC`. The pressure solver calls `EnforcePressureBC`: under the incremental scheme it restores the ring pressure after the projection, where copying the outlet velocity again would undo the projection.

Parameters

- `t` – the time the prescribed values are evaluated at, in seconds.

Example

```
sweepGrid.CopyFlowFields(*gridAnt);
sweepGrid.EnforceVelocityBC(time);    // before the sweep reads its neighbours
```

```
double WallLinkFactor(Component c, Axis a, int i, int j, int k, int s) const;
```

The wall factor f_{wall} (below) of the viscous coupling between a face and its neighbour along a tangential axis. Call it after `EnforceVelocityBC`.

Parameters

- `c` – the velocity component the face belongs to.
- `a` – the axis of the neighbour, tangential to the component.
- `i, j, k` – the face. The 2D form drops `k`.
- `s` – the side of the neighbour, -1 or $+1$.

Returns 1 unless the neighbour is the value of a no-slip or moving wall. For such a neighbour, `FaceGap/(Width/2)` of the face's own cell along `a`. At a wall of the domain this is exactly 2 on every grid, stretched included, because the ring cell is as wide as the interior cell beside it. At the face of an obstacle on a stretched grid it is $(W_f + W_s)/W_f$, with W_f the width of the face's cell and W_s that of the solid cell.

```
void AddForce(double time, double dt);
```

Adds the body force times `dt` to every face that carries fluid, leaving out the faces the boundary sets. The force is the config's `Fu`, `Fv` and `Fw` plus the scene's `force` expression, in units of an acceleration. Nothing happens when the total force is zero.

Parameters

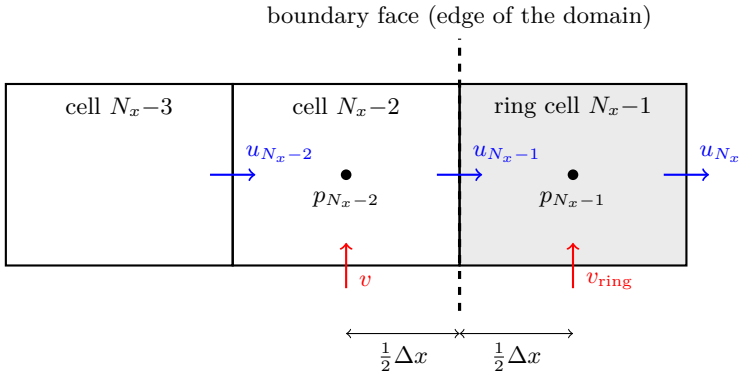
- `time` – the time at which the scene force is evaluated. This is the start of the step.
- `dt` – the time step.

```
void ApplyInitialField();
```

Sets the velocity from the scene's `initial` expression at the staggered points. Nothing happens when the scene has none.

The ring and the boundary face

The figure shows the high end of the x axis in 2D. The last interior cell is N_x-2 and the ring cell is N_x-1. u_{N_x-2} is inside the domain, u_{N_x-1} lies on the boundary face, and u_{N_x} is the outer face of the ring cell. The ring cell's centre, where its pressure and its tangential velocity are stored, is half a cell outside the boundary face. The low end mirrors this, with the ring cell 0, the boundary face u_1 and the outer face u_0.



Velocity

`EnforceVelocityBC(t)` runs four passes in this order.

1. Every face that touches a `SOLID_CELL` gets a zero normal velocity, on the walls of the domain and on every face of an obstacle.
2. The prescribed values are written, from the velocity function evaluated at the face position and at `t`. Along the face's normal the grid writes the boundary face and the outer face (u_1 and u_0 at $x_{\min}$). Along the other axes it writes the ring value (v in ring column 0).
3. The zero-gradient copies are written. Normal components copy inward, u_{N_x-1} from u_{N_x-2} and then u_{N_x} from u_{N_x-1}. A tangential component copies its interior neighbour, v_{ring} from v. Faces are copied first, then edges, then corners, so a corner copies from an edge that is already set.
4. At a continuity outflow the flux is corrected to match the inlets (below).

No-slip and moving walls

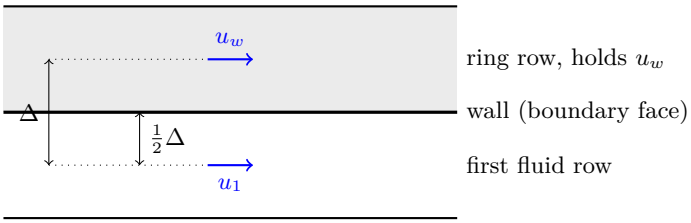
The ring stores the wall velocity u_w at the centre of the ring cell, but the wall is the boundary face, half a cell closer. So the viscous coupling between the first interior face u_1 and the ring is half a cell long. The viscous flux through the wall is

$$\nu \frac{u_w - u_1}{\Delta/2} = \nu \frac{u_g - u_1}{\Delta}, \qquad u_g = 2u_w - u_1,$$

the same as a ghost value u_g reflected through the wall (Harlow and Welch 1965). The coefficient of that coupling is multiplied by

$$f_{\text{wall}} = \frac{\texttt{FaceGap}}{\texttt{Width}/2},$$

which is 2 on a uniform grid. `WallLinkFactor` returns it, and 1 on every coupling that does not end in a no-slip or moving wall, a slip wall included. It holds on the walls of the domain and on every face of an obstacle.



Continuity outflow

After the copies, the outflow faces carry the interior flux, which need not equal the inflow. Let $s = \pm 1$ be the sign of the outward normal of a face, u its normal velocity and A its area. The grid sums

$$Q_{\text{in}} = -\sum_{\text{inlet faces}} s\,u\,A, \qquad Q_{\text{out}} = \sum_{\text{outflow faces}} s\,u\,A, \qquad A_{\text{out}} = \sum_{\text{outflow faces}} A,$$

and adds one uniform correction to every outflow face,

$$\delta = \frac{Q_{\text{in}} - Q_{\text{out}}}{A_{\text{out}}}, \qquad u \leftarrow u + s\,\delta, \qquad u_{\text{outer}} \leftarrow u.$$

The net flux through the boundary is then zero. To the pressure solve an outflow is a known velocity, like an inlet, so with no pressure outlet the pressure system is singular and needs `NEEDS_COMPATIBILITY_CONDITION`.

Pressure

`EnforcePressureBC(t)` sets each ring cell from its type.

- An `EMPTY_CELL`, a pressure outlet without `p=`, gets 0.
- A `PRESSURE_CELL` gets the prescribed $p_b(\mathbf{x}, t)$, evaluated on the boundary face.
- Any other ring cell copies its interior neighbour, a zero normal gradient.

At a pressure outlet. The value p_b belongs on the boundary face, which is half a cell from the centre of the last fluid cell, not a full cell. The pressure matrix and the projection therefore take the gradient over the half cell,

$$\left.\frac{\partial p}{\partial n}\right|_{\text{face}} \approx \frac{p_b - p_{\text{in}}}{\Delta/2},$$

which multiplies the coefficient of that face by $f_{\text{D}} = \texttt{FaceGap}/(\texttt{Width}/2)$, 2 on a uniform grid. A linear pressure field is then exact to round-off. The outlet's normal velocity is first copied from inside; the projection then corrects it with this gradient, so the flux through the outlet face is divergence free.

At a wall or an inlet. Here the normal velocity g_n is known and the pressure is not. Take the normal component of the momentum equation on the boundary face, with the derivatives along the face set to zero. Continuity then gives $\partial u_n/\partial n = 0$, so the convective and the viscous terms vanish, and what the velocity prescribes for the pressure is

$$\frac{\partial p}{\partial n} = \rho \left(f_n - \frac{\partial g_n}{\partial t} \right),$$

the hydrostatic gradient at a wall at rest. The code does not impose this. The boundary face's u^* is set to $g_n(t^{n+1})$ and the projection does not correct it, so the solve uses the homogeneous condition

$$\frac{\partial p}{\partial n} = 0$$

for the projection's pressure (Chorin) or increment (incremental). The force and $\partial g_n/\partial t$ enter u^* on the interior faces, so the right gradient is recovered inside the domain; the difference is the $O(\Delta t)$ artificial boundary layer of projection methods (Guermond, Minev and Shen 2006). A `pressure_inlet` kind, which would prescribe the pressure and let the normal velocity follow, is not implemented.

4.8 Differential operators

This part of the interface is small and is being reworked. A future version will offer the gradient, the divergence and the Laplacian of any field, and a gradient at any physical point, in one consistent set of functions. Until then a method writes its own stencils from the spacing tables, as the tutorial does.

```
double GetDivergencyAt(int i, int j, int k);
```

The discrete divergence of a cell, the net flux through its faces divided by its volume.

Parameters

- `i, j, k` – the cell. The 2D form drops `k`.

Returns the divergence, in 1/s. It is zero for a cell that is not fluid.

```
double GetGradP(Axis a, int i, int j, int k);
double GetGradP(Axis a, int i, int j);
```

The pressure gradient on the low face of a cell along an axis, where `u`, `v` or `w` sits: the difference of the two cell pressures divided by `FaceGap`, the same d_f the pressure matrix uses.

Parameters

- `a` – the axis of the face.
- `i, j, k` – the cell whose low face it is. The 2D form drops `k`.

Returns the gradient, in Pa/m. It is not defined on the outermost faces.

4.9 Interpolation and sampling

```
double ComponentAtFace(Component c, Component target, int i, int j, int k);
double getVatU(int i, int j, int k);   // also getWatU, getUatV, getWatV, getUatW, getVatW
```

One velocity component where another is stored. It averages four values, over the two faces of `c` along its axis and the two cells along the axis of `target`, weighted by distance on a stretched grid. The `getXatY` forms are short names, so `getVatU` is `v` at a `u` face.

Parameters

- `c` – the component to sample.
- `target` – the component whose face `(i, j, k)` is the sample point.
- `i, j, k` – the face. The 2D form drops `k`.

Returns the interpolated value.

```
double ComponentAt(Component c, double x, double y, double z);
```

A field at a physical point, by trilinear interpolation (bilinear in 2D). A velocity component is interpolated over its own faces along its axis and over the nearest cell centres along the others. The pressure is interpolated over the cell centres. The interpolation does not look at the cell types, so a caller that must not read a solid checks for one itself.

Parameters

- `c` – `MAC::U`, `V`, `W` or `P`.

- `x, y, z` – the point, in metres. The 2D form drops `z`.

Returns the interpolated value. Beyond the first and the last centre it is constant.

```
double ScalarAt(int handle, double x, double y, double z);
double ScalarAt(const std::string &name, double x, double y, double z);
```

A registered scalar at a point, sampled in the same way as the pressure.

Parameters

- `handle, name` – the scalar, by its handle or by its name.
- `x, y, z` – the point, in metres.

Returns the interpolated value, or 0 for a scalar the grid does not carry.

Example

```
const double p = g.ComponentAt(MAC::P, 0.45, 0.20, 0.205);
const double d = g.ScalarAt("d", 0.45, 0.20, 0.205);
```

```
int CellAt(Axis axis, double x) const;
```

The cell that holds a coordinate along one axis. There is no single call from a point to `(i, j, k)`: call it once per axis, `MAC::X` for `j`, `MAC::Y` for `i` and `MAC::Z` for `k`. `CentrePos` goes back from an index to a coordinate.

Parameters

- `axis` – the axis.
- `x` – the coordinate, in metres.

Returns the cell index along that axis. A coordinate outside the grid gives the first or the last cell.

Example

```
const int j = g.CellAt(MAC::X, x), i = g.CellAt(MAC::Y, y);   // the cell holding the point (x, y)
const double p = g.GetP(i, j);
```

4.10 Diagnostics

```
bool AllFinite() const;
```

Checks every value of the velocity and the pressure. The shipped program calls it after every step and ends a run whose field is not finite.

Returns true when all of them are finite.

```
double GetMaxVelocity();
```

The largest speed of the velocity, taken at the cell centres. A NaN does not change it, so use `AllFinite` to detect a blown-up field.

Returns the speed, in m/s.

```
double GetDivSum();
```

The sum of the absolute divergence over the interior cells, the divergence the telemetry reports.

Returns the sum.

```
int GetFluidCellCount();
```

Returns the number of fluid cells.

```
double MaxAbsoluteDifference(MAC &grid);
```

The largest change in the velocity between this grid and another, the step residual.

Parameters

- `grid` – a grid of the same size.

Returns the largest absolute difference in `u`, `v` or `w`.

4.11 Scalar registry

A grid can carry cell-centred scalars, such as a density, a temperature or a tracer. Each one is registered by name. It is sized and indexed like `p`. Copies, checkpoints and exports carry it.

```
int RegisterScalar(const std::string &name, TransportScheme scheme = TransportScheme::Upwind);
int RegisterScalar(const std::string &name, const ScalarTransport &transport);
```

Adds a field of zeros. A second call with the same name returns the same handle and keeps the first transport, and a different transport in the second call is reported.

Parameters

- `name` – the name of the field.
- `scheme, transport` – how the transport solver moves it: the scheme alone, or the solver, scheme, form, diffusivity and reaction rate. A `transport.solver` of `ScalarSolver::None` leaves the field to the method: the transport step only sets its ring.

Returns the handle of the field.

Example

```
int d = g.RegisterScalar("d");
g.SetS(d, i, j, k, 1.0);
```

```
int ScalarHandle(const std::string &name) const;
```

Parameters

- `name` – the name of the field.

Returns the handle of the field, or -1 when the grid does not carry it.

```
int ScalarCount() const;
const std::string &ScalarName(int h) const;
```

Parameters

- `h` – a handle, from 0 to `ScalarCount() - 1`.

Returns the number of fields, and the name of field `h`.

```
const ScalarTransport &ScalarTransportOf(int h) const;
```

Parameters

- `h` – the handle of the field.

Returns the field's solver, scheme, form, diffusivity and reaction rate.

```
void EnforceScalarBC(int h, double t);
```

Sets the ring of field `h`: on a face where the scene pins the scalar, the scene's value at time `t`; on every other ring cell, a copy of the cell inside (zero gradient). The transport step calls it after each solve, and for a field with no solver (`solver=none`) it is all the step does, so a method that updates such a field itself (a turbulence model, say) finds the boundary values in the ring.

Parameters

- `h` – the handle of the field.
- `t` – the time the scene's value is evaluated at.

```
double ScalarSourceAt(int h, double x, double y, double z, double t) const;
```

Evaluates field `h`'s `source` expression from the scene. The ADI transport solver applies it once per step, before its sweeps, at each fluid cell centre; a method updating a field with no solver calls this to do the same. Call it from serial code only: the expression's variables live at fixed addresses, so two threads evaluating it at once overwrite each other's.

Parameters

- `h` – the handle of the field.
- `x, y, z` – the point, in metres. In 2D `z` is ignored.
- `t` – the time, in seconds.

Returns $S(x, y, z, t)$, or 0 when the field has no source.

```
double GetS(int h, int i, int j, int k) const;
void SetS(int h, int i, int j, int k, double value);
```

Read or write one value of a field, at a cell centre.

Parameters

- `h` – the handle of the field.
- `i, j, k` – the cell. The 2D form drops `k`.
- `value` – the value to store.

Returns the stored value (`GetS`).

```
VectorXd &ScalarField(int h);
```

Parameters

- `h` – the handle of the field.

Returns a reference to the whole array of the field, indexed like `p`.

```
void ClearScalars();
```

Removes every registered field.

5 Methods, settings and steps (Definitions.h)

`src/headers/Core/Definitions.h` holds the types a method author needs: the method row, the capability bits, the settings tables and the step list.

5.1 The method row

```
struct SimulationMethod {
    const char *key;
    const char *label;
    const char *help;
    MethodFamily family;
    bool supports2D;
    bool supports3D;
    unsigned caps;
    void (*initialize)();
    void (*dispose)();
    void *settings;
    const SettingField *settingFields;
    int settingCount;
    void (*drawSettings)(const SettingsContext &ctx);
    void (*checkSettings)(const SettingsContext &ctx, std::vector<std::string> &problems,
                          std::vector<std::string> &notes);
    double (*stableTimestep)();
    double (*maxVelocity)();
    double (*cflLength)();
```

```
    double (*particleSpacing)();
};
```

Everything the program knows about a method. `key` is the value of `TYPE` in the config, the prefix of its settings block (`<KEY>_BEGIN`), and the tag in a checkpoint. It may be 1 to 63 characters long (`MAX_METHOD_KEY` −1), the size of the checkpoint's key field. The setup screen shows `label` and `help`. `initialize` builds the method's solvers and registers its steps (Section 5.3). `dispose` frees what `initialize` allocated; it may be null when the method holds nothing worth freeing. The settings members point at the method's settings struct and at its table (Section 5.2). `drawSettings` and `checkSettings` are optional hooks for the setup screen. The last four members are optional questions. A null pointer means the manager answers from the grid.

Parameters

- `family` – `MethodFamily::EULERIAN`, `HYBRID` or `LAGRANGIAN`. The family groups the method on the setup screen and limits its capabilities.
- `caps` – the capability bits below, combined with OR. They must stay within what the family offers.
- `stableTimestep` – the method's own largest stable step. It is used under `ADAPTATIVE_TIMESTEP`.
- `maxVelocity, cflLength` – the speed and the length for the CFL number, for a method whose grid does not carry them.
- `particleSpacing` – the distance between seeded particles. The surface export uses it.

```
enum MethodCapability : unsigned {
    CAP_GRID, CAP_PARTICLES, CAP_RECTILINEAR, CAP_BOUNDARY_CELLS,
    CAP_FIXED_REGION, CAP_SCALARS, CAP_PRESSURE_SOLVER };
```

Each capability is one bit of a mask, so a row lists what it supports by combining them with `|`.

- `CAP_GRID`. A MAC grid carries the solution, so there are field views and `.vtr` exports.
- `CAP_PARTICLES`. There is a particle set to draw and export.
- `CAP_RECTILINEAR`. The method can run on a stretched grid.
- `CAP_BOUNDARY_CELLS`. The border is a ring of boundary cells.
- `CAP_FIXED_REGION`. The fluid region does not move.
- `CAP_SCALARS`. The registered scalars are transported.
- `CAP_PRESSURE_SOLVER`. The method uses the shared pressure solver (Section 6), so the setup screen shows the settings of that solver.

Example

```
// An Eulerian method on stretched grids, with the shared pressure solver and no scalars
const unsigned caps = CAP_GRID | CAP_RECTILINEAR | CAP_BOUNDARY_CELLS | CAP_FIXED_REGION | CAP_PRESSURE_SOLVER
    ;
if (caps & CAP_SCALARS) { /* never: the bit is not set */ }
```

A family is an upper limit. A row that claims a bit its family does not offer is refused when it is registered.

family	grid	particles	rectilinear	boundary cells	fixed region	scalars	shared PPE
`EULERIAN`	yes		yes	yes	yes	yes	yes
`HYBRID`	yes	yes		yes			yes
`LAGRANGIAN`		yes			yes		

```
int RegisterSimulationMethod(const SimulationMethod &m);
```

Adds a row to the registry. It stops with an error on a key that is empty, longer than 63 characters or already used, on capabilities outside the family, or when the registry is full (16 rows). Call it before `InitializeSimulation`, which looks up the config's `TYPE` here.

Parameters

- `m` – the method's row.

Returns the id of the row. `SIM_TYPE` holds this id while the method runs.

```
const SimulationMethod &MethodOf(int id);
```

Parameters

- `id` – the id `RegisterSimulationMethod` returned.

Returns the row. An id out of range gives an empty `UNKNOWN` row, so the lookup never fails.

```
const SimulationMethod &ActiveMethod();
```

Returns the row of the method the config chose, `MethodOf(SIM_TYPE)`.

Example

```
std::printf("running %s\n", ActiveMethod().key);
```

```
int MethodFromKey(const std::string &key);
```

Parameters

- `key` – a method's key, the value of `TYPE` in a config.

Returns the id of the row, or −1 for an unknown key.

```
bool MethodSupports(int id, unsigned cap);
```

Parameters

- `id` – the id of a row.
- `cap` – one capability bit, `CAP_GRID` to `CAP_PRESSURE_SOLVER`.

Returns true when the row sets the bit and its family allows it.

```
bool MethodUsesGrid();
bool MethodUsesParticles();
bool MethodSupportsRectilinear();
bool MethodUsesBoundaryCells();
bool MethodFixedFluidRegion();
```

Shorthands for `MethodSupports` on the active method, one capability each. Shared code uses them to decide what to do without naming a method.

Returns true when the active method has the capability.

```
double ActiveMethodSetting(const char *key, double fallback);
```

Reads a setting of the active method by its config key. Shared code uses it so that it does not have to name a method.

Parameters

- `key` – the config key of the setting.
- `fallback` – the value to return when the active method has no such key.

Returns the value of the setting, or `fallback`.

5.2 Settings tables

A method's settings are a plain struct with one global instance, in the method's own header. A table of `SettingField` rows says where each field sits and what its config key is. The config reader, the config writer and the setup screen all walk this table, so each key is written in one place only.

```
constexpr SettingField IntSetting(const char *key, const char *label, const char *help,
                                  size_t offset, int def,
                                  double lo = SETTING_NO_MIN, double hi = SETTING_NO_MAX);
constexpr SettingField DoubleSetting(const char *key, const char *label, const char *help,
                                     size_t offset, double def,
                                     double lo = SETTING_NO_MIN, double hi = SETTING_NO_MAX);
constexpr SettingField BoolSetting(const char *key, const char *label, const char *help,
                                   size_t offset, bool def);
```

```
constexpr SettingField EnumSetting(const char *key, const char *label, const char *help,
                                   size_t offset, int def,
                                   const char *const *keys, const char *const *labels, int count);
```

Each function makes one row. The setup screen draws an integer with both bounds as a slider, a double as an input box kept inside its range, a bool as a checkbox and an enum as a drop-down list. `help` is the tooltip.

Parameters

- `key` – the config key. It is read inside the method's block, between `<KEY>_BEGIN` and `<KEY>_END`.
- `label, help` – the name and the tooltip on the setup screen.
- `offset` – `offsetof(TheStruct, field)`.
- `def, lo, hi` – the default and the range.
- `keys, labels, count` – how each enum value is written in the file and shown on screen, in enum order.

Returns the row, to put in the method's table.

```
constexpr SettingField WithUnit(SettingField f, const char *unit);
```

The same row with a unit, which the setup screen shows after the value. Rows without one show none.

Parameters

- `f` – a row made by one of the functions above.
- `unit` – the unit as it should read, `"m"`, `"m/s"`, `"1/s"`.

Returns the row with its unit set.

Example

```
struct MySettings { double relaxation = 1.7; int maxIterations = 1000; double radius = 0.05; };
inline MySettings MY_SETTINGS;
inline const SettingField MY_FIELDS[] = {
    DoubleSetting("MY_RELAXATION", "Relaxation", "SOR factor, 0.5 to 2",
                  offsetof(MySettings, relaxation), 1.7, 0.5, 2.0),
    IntSetting("MY_MAX_ITERATIONS", "Max sweeps", "", offsetof(MySettings, maxIterations), 1000, 1, 1e7),
    WithUnit(DoubleSetting("MY_RADIUS", "Radius", "", offsetof(MySettings, radius), 0.05, 0.0), "m"),
};
```

```
struct SettingsContext {
    int dimension;
    double dh, density, viscosity, timeStep;
    bool adaptiveTimestep;
    double x0, xf, y0, yf, z0, zf, fu, fv, fw, meanVelocity, characteristicLength;
};
```

What a method's panel or check is told about the rest of the setup. A panel uses it for read-outs such as a cell Reynolds number.

5.3 Steps

```
using SimulationStepFunction = void (*)(MAC *gridAnt, MAC *gridSol,
                                        ParticleSet *particles, double time);
using SimulationStepCounter  = int (*)();
void RegisterSimulationStep(SimulationStepFunction function, const std::string &name, int order,
                            SimulationStepCounter counter = nullptr);
```

Adds a step to the list the manager runs every time step. The list is sorted by `order`. Steps with the same order keep their registration order. A method's `initialize` calls it. The manager times each step and writes one telemetry column `<name>_timing` per step, so a step needs no timing code.

The optional `counter` lets the manager learn a step's iteration count without asking any method for it. After the step runs, the manager calls it and writes the value in the column `<name>_counter`. A step registered without one reports −1. The pressure solver's `pressure` step returns the iterations of its last solve. The counter columns are written with the timing columns, when `EXPORT_COLUMN_STEP_TIMINGS` is on.

Parameters

- `gridAnt` – the field at the start of the step. Read it. Do not write it.
- `gridSol` – the field the step builds. The manager zeroes it before the first step.
- `particles` – the particle set of the run. It is empty for a method without particles.
- `time` – the simulated time at the start of the step. `SIMULATION.time` holds the time the step solves for.
- `order` – the position in the list. The shipped steps use the orders in the table below.
- `counter` – a function that returns the step's work count for the step it just ran, or `nullptr`.

A step works only on the grids and particles it is handed, and writes no global state. The manager measures the divergence after the last step.

order	name	registered by	method
0	`momentum`, `particle_advection`, `sph`	the method	ADI, FLIP, SPH
2	`grid_diffusion`	the method	FLIP
4	`pressure`	the pressure solver	ADI, FLIP
6	`projection`	the pressure solver	ADI, FLIP
7	`grid_to_particle`	the method	FLIP
8	`density_correction`	the method, when it is on	FLIP
8	`transport`	the transport solver, when there are scalars	ADI

6 The pressure solver (`Methods/PPESolver/`)

`PressureSolver2D` and `PressureSolver3D`, with `PressureSettings.h`. A shared solver for the pressure Poisson equation that any grid method can use. It is not a method and registers no row. The method tells it which projection scheme it wants, and the solver does the rest: it registers its own steps, builds the matrix from the cell types, keeps the matrix while the cells do not change, and solves on the CPU (Eigen or AMGCL) or on an NVIDIA GPU (AMGX). The config's `PRESSURE` block chooses the backend. A method that sets `CAP_PRESSURE_SOLVER` uses this solver.

The solve finds the pressure from

$$\nabla^2 p = \frac{\rho}{\Delta t} \nabla \cdot \mathbf{u}^*,$$

and the projection corrects the velocity on every face that is not a wall,

$$\mathbf{u} = \mathbf{u}^* - \frac{\Delta t}{\rho} \nabla p.$$

Walls, inlets and outflows give a Neumann condition, a pressure outlet a Dirichlet condition on the boundary face (Section 4.7).

```
static void InitializePressureSolver(MAC *grid, PressureScheme scheme = PRESSURE_CHORIN);
```

Chooses the backend, builds the matrix, and registers the `pressure` step (order 4) and the `projection` step (order 6). Call it once from the method's `initialize`. The `pressure` step is registered with the counter `LastIterations`, so the telemetry column `pressure_counter` holds the iterations of each solve.

Parameters

- `grid` – the grid whose cell types define the system.
- `scheme` – `PRESSURE_CHORIN` or `PRESSURE_INCREMENTAL`. With Chorin the solve returns the pressure. With the incremental scheme the momentum step includes the old pressure gradient, the solve returns the increment, and the projection adds it to the old pressure.

```
static int LastIterations();
```

The `pressure` step's counter (Section 5.3).

Returns the iterations of the last pressure solve.

```
static void SolveChorin(MAC *gridAnt, MAC *gridSol, ParticleSet *particles, double time);
static void SolveIncremental(MAC *gridAnt, MAC *gridSol, ParticleSet *particles, double time);
static void SolvePressure(MAC *gridAnt, MAC *gridSol, ParticleSet *particles, double time);
static void Project(MAC *gridAnt, MAC *gridSol, ParticleSet *particles, double time);
```

`SolveChorin` and `SolveIncremental` are the two pressure steps; `InitializePressureSolver` registers one of them. `SolvePressure` runs the one it chose, for a program that drives the solver itself. `Project` is the projection step. Under the incremental scheme it also forms the new pressure.

Parameters

- `gridAnt, gridSol, particles, time` – the arguments of every step (Section 5.3). The solve reads the divergence of `gridSol` and writes the pressure there. The incremental scheme reads the old pressure from `gridAnt`.

```
static bool SolveLaplacian(MAC *grid, const VectorXd &f, VectorXd &x);
static int SystemSize();
```

`SolveLaplacian` solves $Lx = f$ on the current fluid cells, with the boundary conditions of the pressure matrix. It reuses the matrix and its multigrid hierarchy, for a second Poisson solve in the same step. FLIP's density correction uses it. `SystemSize` is the length of the vectors it takes.

Parameters

- `grid` – the grid whose fluid cells define the system.
- `f` – the right-hand side, one value per fluid cell.
- `x` – the solution, written in place.

Returns false when the solve did not converge (`SolveLaplacian`), the number of unknowns (`SystemSize`).

```
static void ReleasePressureSolver();
static void DestroyPressureSolver();
```

`ReleasePressureSolver` frees the matrix, the AMG hierarchy and the AMGX handle, and leaves the AMGX library up; a method's `dispose` calls it, so a later run can use AMGX again. `DestroyPressureSolver` does the same and then shuts AMGX down when no handle is left. A program that used AMGX calls it once, at exit.

7 Scalar transport

`TransportSolver2D` and `TransportSolver3D`, declared in `src/headers/Solvers/` and defined in `src/`. They move every scalar ϕ the grid carries with the solved velocity, with optional diffusion, reaction and source,

$$\frac{\partial \phi}{\partial t} + \mathrm{conv}(\mathbf{u}, \phi) = \nabla \cdot (\epsilon \nabla \phi) - \gamma \phi + S.$$

A scalar is declared in the scene (Section 10) or registered with `MAC::RegisterScalar`. Its solver, its scheme (central or upwind), its form (flux or advective), ϵ and γ travel with it. A scalar declared with `solver=none` is stored, copied, checkpointed and exported like the others, and its ring is set every step, but the transport step does not move it: the method that owns it updates the values.

```
static void InitializeTransportSolver2D(MAC *grid);
static void InitializeTransportSolver3D(MAC *grid);
```

Sizes the work arrays and registers the `transport` step at order 8, after the projection, so the scalars move with the divergence-free velocity. Call it from the method's `initialize` when the grid carries scalars.

Parameters

- `grid` – the grid whose scalars are transported.

```
static void SolveTransportStep(MAC *gridAnt, MAC *gridSol, ParticleSet *particles, double time);
```

The `transport` step. It moves every registered scalar one time step.

Parameters

- `gridAnt` – the grid the scalars are read from.
- `gridSol` – the grid the new scalars are written to, with the projected velocity that moves them.
- `particles, time` – the remaining step arguments. `time` is the start of the step.

8 Particles

`Definitions.h` holds the particle types. `src/headers/Core/Particles.h` holds the operations that every particle method shares. Particle runs are limited to closed domains with fixed walls: no particle is injected or removed at a boundary, and every wall is stationary for the contact (Section 4.7).

```
struct Particle { bool isActive; double x, y, z; double u, v, w; };
struct ParticleSet { std::vector<Particle> data; int count; std::vector<double> density; };
```

The particles of a run live in `SIMULATION.particles`, and every step receives them. `data.size()` is the capacity, set once by the method's `initialize`. The first `count` entries are the live particles. In 2D, `z` and `w` stay zero. `density` is for a method that carries one density per particle between steps (SPH); it is empty otherwise, and the checkpoint writes it when it is not. Any other data a method needs stays in the method.

```
void Particles::ResolveContacts(MAC *grid, const Particle &previous, Particle &p);
```

Keeps a particle of a grid method out of the walls and the obstacles after it has moved, and applies the contact response (Section 8.1). Section 8.2 gives the stages.

Parameters

- `grid` – the grid whose cell types mark the solids.
- `previous` – the particle before the move.
- `p` – the particle after the move, corrected in place.

Example

```
const Particle before = p;
p.x += p.u * dt;  p.y += p.v * dt;
Particles::ResolveContacts(grid, before, p);
```

```
void Particles::ResolveContactsAnalytic(Particle &p, double margin);
```

The same for a gridless method, against the scene's exact surface (Section 8.3).

Parameters

- `p` – the particle, corrected in place.
- `margin` – the distance kept from every surface, in metres. SPH sets it from its kernel radius.

```
void Particles::WallBox(double lo[3], double hi[3]);
```

The box the walls of the domain enclose, as particles see it. The low corner is the low face of each axis. The high corner is the last interior face, which is the domain's high face unless its length is not a whole number of cells. `ClampToDomain` keeps particles inside it, and the liquid surface export clips the surface to it, so the two agree.

Parameters

- `lo, hi` – set to the low and the high corners, in metres.

```
void Particles::ClampToDomain(Particle &p);
```

The wall stage of `ResolveContacts`. A particle on or beyond a face of `WallBox` is put back on it and gets the contact response with that wall's inward normal.

Parameters

- `p` – the particle, corrected in place.

```
int Particles::SubstepCount(double speed, double dt);
```

How many substeps a move needs so that no substep covers more than about one cell. A contact test only sees where a move ends, so a particle that crosses a thin obstacle in one move would not be caught.

Parameters

- `speed` – the particle's speed, in m/s.

- `dt` – the time step, in seconds.

Returns the number of substeps,

$$n_{\text{sub}} = \min\Big(1 + \Big\lfloor \frac{|\mathbf{v}|\,\Delta t}{\Delta} \Big\rfloor,\ 8\Big).$$

```
double Particles::Speed(const Particle &p);
```

Parameters

- `p` – the particle.

Returns the particle's speed, in m/s.

8.1 Contact response

A particle that meets a wall or an obstacle keeps its position on the fluid side, and its velocity changes. Let $\mathbf{n}$ be the outward unit normal of the surface and v_n the normal velocity, $\mathbf{v} \cdot \mathbf{n}$. Only a particle moving into the surface ($v_n < 0$) is changed. The config key `PARTICLE_CONTACT` picks the response.

Response	New velocity	Meaning
`SLIP` (default)	$\mathbf{v} - v_n\,\mathbf{n}$	the normal part is removed; the particle slides along the surface
`NOSLIP`	$\mathbf{0}$	the particle stops
`BOUNCE`	$\mathbf{v} - (1+e)\,v_n\,\mathbf{n}$	the normal part is reflected and scaled by the restitution e between 0 and 1 (`PARTICLE_RESTITUTION`); 0 is a slip, 1 a mirror reflection

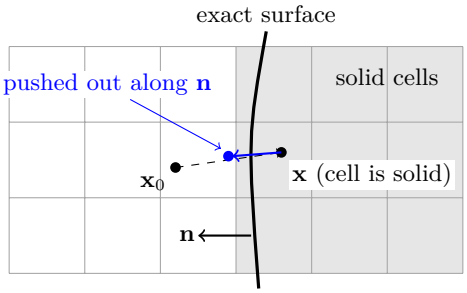
8.2 Contact on grid methods

`ResolveContacts` works in three stages, walls, then obstacles, then walls again, so an obstacle that touches a wall cannot push a particle out through the wall. The walls are `ClampToDomain`.

Obstacles. The voxel mask decides whether a particle is inside an obstacle, because the mask is what the pressure solve sees. A particle whose cell is not `SOLID_CELL` is left alone. Otherwise the scene gives the signed distance ϕ (negative inside) and the outward normal $\mathbf{n}$, and the particle moves out along the normal,

$$\mathbf{x} \leftarrow \mathbf{x} + \big(-\phi + \epsilon\,\Delta\big)\,\mathbf{n},$$

where ϵ is `PARTICLE_SURFACE_OFFSET` in cells (default 0.01). If its cell is still solid, which happens because the voxel solid can reach up to a cell beyond the exact surface, it walks along $\mathbf{n}$ in steps of a quarter cell, at most 8, until it is out. The contact response is then applied with $\mathbf{n}$. If the normal is not defined (on a medial axis) or the walk does not get out, the particle goes back along its move to the last point outside the mask.



8.3 Contact on gridless methods

SPH has no voxel mask, so `ResolveContactsAnalytic` uses the scene's exact surface alone and keeps a standoff `margin` from every surface.

- A particle closer than `margin` to a face of the domain is put at that distance and gets the response with the wall's inward normal.
- A particle inside a solid moves by $(-\phi + \text{margin})\,\mathbf{n}$ and gets the response with $\mathbf{n}$. On a medial axis it is left alone.
- The order is walls, obstacles, walls, as on a grid.

9 The Configuration file

One config file describes a run. It holds `KEY = value` lines, then four kinds of block: `PRESSURE_BEGIN` to `PRESSURE_END` for the shared pressure solver, one `<KEY>_BEGIN` to `<KEY>_END` block per registered method, and `SCENE_BEGIN` to `SCENE_END` for the geometry (Section 10). A `#` starts a comment, also after a value. A key that is left out takes the default below. The setup screen writes every key, so a file it saves lists them all.

Validation. `InitializeSimulation` reads the whole file before it builds anything and stops the program with exit code 1, listing every problem at once, when

- the file does not exist (`configuration file '...' not found`);
- a key is not read by anything: a misspelling at the top level, or a key that is not in the table of the method block it sits in;
- a number does not parse in full (`VISCOSITY = abc` or `0.01abc`), or an integer key holds a real (`GRID_SIZE = 32.5`);
- a choice is outside its list (`PROJECTION_SCHEME = FOO`), with the allowed values printed;
- a `REFINE_*` interval does not parse or is empty;
- a `<KEY>_BEGIN` block belongs to a method the program did not register.

The pressure block is checked by its backend (AMGX parses its own keys), and the scene block by the scene parser (Section 10). The setup screen opens the same file without stopping: it keeps the default for a value it cannot read, and writes every key back, valid, when the case is saved.

9.1 The run

Key	Default	Meaning
`DIMENSION`	3	2 or 3.
`TYPE`	the first method registered	the key of the method to run, for example `ADI`, `FLIP`, `SPH` or `EXPLICIT`. A key that names no registered method stops the run with an error.
`THREAD_COUNT`	4	OpenMP threads.
`GPU_ACCELERATION`	0	1 solves the pressure with AMGX. Forced to 0 in a build without the GPU.

9.2 Domain and grid

Key	Default	Meaning
DOMAIN_XO, DOMAIN_XF, DOMAIN_YO, DOMAIN_YF, DOMAIN_ZO, DOMAIN_ZF	0	the extents of the domain, in metres. Always set them.
GRID_SIZE	16	cells inside the domain across the shorter of the x and y lengths. The spacing is that length divided by GRID_SIZE, the same on every axis; the ring adds one cell outside each face.
NON_UNIFORM_GRID	0	1 enables the refinement below. Only a method with CAP_RECTILINEAR honours it; any other is given a uniform grid, with a warning.
REFINE_X, REFINE_Y, REFINE_Z	none	refined intervals, separated by ;, each `start end factor [mode]`. Mode `hard` (default) splits each cell into `factor` cells; `ramp` grows the factor from 1 at `start` to `factor` at `end`; `ramp_back` is its mirror; `graded` is 1 at both ends and `factor` in the middle. Where intervals overlap the finest wins. An interval outside the domain refines nothing. Section 4.3 gives the construction.

```
NON_UNIFORM_GRID = 1
REFINE_X = 0 0.1 4 ramp ; 0.1 0.3 4 hard ; 0.3 0.7 4 ramp_back
```

9.3 Fluid and reference values

Key	Default	Meaning
VISCOSITY	0.01	kinematic viscosity ν, in m^2/s.
DENSITY	1	density ρ, in kg/m^3.
Fu, Fv, Fw	0	a uniform body force, as an acceleration in m/s^2 (gravity is `Fv = -9.81`). A force that varies in space or time is the scene's `force` directive.
MEAN_VELOCITY	1	the reference speed U of the force coefficients.
CHARACTERISTIC_LENGTH	0.1	the reference length L of the force coefficients.
CHARACTERISTIC_LENGTH_2	the value of L	in 3D the coefficients divide by the area $L \cdot L_2$ (diameter times span, chord times span).
REYNOLDS_NUMBER	0	used only in the name of the output folder. The flow's Reynolds number follows from VISCOSITY, MEAN_VELOCITY and CHARACTERISTIC_LENGTH.

9.4 Time stepping and the end of the run

Key	Default	Meaning
TIME_STEP	0	the time step Δt, in seconds. Always set it. Under ADAPTATIVE_TIMESTEP it is the largest step allowed.
ADAPTATIVE_TIMESTEP	0	1 sets Δt every step from MAX_CFL, or from the method's own stable step when its row gives one.
MAX_CFL	1	the target advective CFL number $\Delta t\,\lvert u\rvert_{\max}/h$.
STOP_CONDITION	NEVER	NEVER, TIME or RESIDUAL. A run without a window refuses NEVER.
STOP_TIME	1	under TIME: the run ends exactly at this time, in seconds; the last step is shortened to land on it.
STOP_RESIDUAL	10^{-6}	under RESIDUAL: the run ends when the step residual (Section 11) falls below this.
STOP_EXPORT	1	1 writes the final field and telemetry row when the rule fires.

9.5 Output

Key	Default	Meaning
EXPORT_ENABLED	1	0 writes nothing.
EXPORT_BASE_PATH	Exports	the folder the run's folder is made in.
EXPORT_TELEMETRY_INTERVAL	1	a telemetry row every this many steps.
EXPORT_FRAME_MODE	ITERATIONS	when frames are written: every EXPORT_GRID_INTERVAL steps, or under TIME every EXPORT_FRAME_INTERVAL seconds of simulated time.
EXPORT_GRID_INTERVAL	100	steps between frames.
EXPORT_FRAME_INTERVAL	0.04	seconds between frames.
EXPORT_GRID	0	1 writes the field as VTK/*.vtr: velocity, pressure, cell type and the scalars.
EXPORT_PARTICLES	0	1 writes the particles as CSV (particle methods).
EXPORT_SURFACE	0	1 writes the liquid surface as PLY (3D particle methods).
EXPORT_SURFACE_SOLIDS	1	with EXPORT_SURFACE: also the obstacles, once, as their own PLY.
EXPORT_SURFACE_VOXEL_SCALE, _RADIUS_SCALE, _INFLUENCE_SCALE	0.75, 1, 2	the surface's lattice pitch, particle radius and averaging radius, in particle spacings. A larger influence radius gives a smoother surface.
EXPORT_SURFACE_SMOOTHING	2	smoothing passes over the surface's distance field.
EXPORT_VOLUMETRIC	0	1 writes each scalar as a NumPy array per frame, with a JSON file of the grid.
EXPORT_AERODYNAMICS	0	1 computes and writes the force coefficients.
EXPORT_DAMBREAK	0	1 computes and writes the free-surface height and front.
EXPORT_COLUMN_*	1	one flag per group of telemetry columns (Section 11).

9.6 The pressure block

The block configures the shared pressure solver (Section 6). It is written in every file and read only by a method that uses that solver.

```
PRESSURE_BEGIN
NEEDS_COMPATIBILITY_CONDITION = 1   # a pressure fixed only up to a constant (no pressure outlet)
PRESSURE_WARM_START = 0
eigen_solver = AMG                  # CPU: AMG (AMGCL), CG or BICGSTAB
eigen_tolerance = 1e-06
config_version=2                    # everything else is AMGX's own configuration (GPU)
solver=PCG
tolerance=1e-06
...
PRESSURE_END
```

Key	Default	Meaning
NEEDS_COMPATIBILITY_CONDITION	0	1 makes a singular system solvable: when no face fixes the pressure (every boundary a wall, an inlet or an outflow) the right-hand side's mean is removed. Required in that case.
PRESSURE_WARM_START	0	1 starts each Chorin solve from the previous pressure.
eigen_solver	BICGSTAB	the CPU solver: AMG (CG with AMGCL's algebraic multigrid, the fastest), CG or BICGSTAB.
eigen_preconditioner	DIAGONAL	for CG and BICGSTAB: DIAGONAL, IDENTITY, ILUT or IC.
eigen_tolerance	10^{-9} (2D), 10^{-12} (3D)	the relative residual the CPU solve stops at.
eigen_max_iters	2000	the CPU solve's iteration cap.
any other line		passed to AMGX as its configuration, see the AMGX reference. On a singular system set amg_solver:coarse_solver=NOSOLVER; AMGX's default dense coarse solve fails on it.

9.7 Particle options

Read by every method with particles.

Key	Default	Meaning
PARTICLE_CONTACT	SLIP	the contact response, SLIP, NOSLIP or BOUNCE (Section 8.1).
PARTICLE_RESTITUTION	0.3	the restitution e under BOUNCE.
PARTICLE_SURFACE_OFFSET	0.01	how far outside a surface a pushed-out particle is put, in cells.
PARTICLE_SEED_JITTER	0	a random offset of each seeded particle, as a fraction of its spacing.

9.8 The method blocks

Each method reads its own block, ADI_BEGIN to ADI_END and so on. The setup screen writes the blocks of every registered method, so switching TYPE keeps the others' settings.

ADI

Key	Default	Meaning
CONVECTION_SCHEME	CENTRAL	CENTRAL (second order) or UPWIND (first order, bounded).
CONVECTION_FORM	FLUX	FLUX or ADVECTIVE.
PROJECTION_SCHEME	CHORIN	CHORIN or INCREMENTAL.
VORTICITY_CONFINEMENT	0	1 adds a force that keeps small vortices from decaying. A graphics aid, not physics; no validated case uses it, and it will be removed in a future version.
VORTICITY_EPSILON	10	its strength, in 1/s.
VORTICITY_SOLID_MARGIN	2	no confinement within this many cells of a solid.

FLIP

Key	Default	Meaning
PARTICLES_PER_CELL	4	particles seeded per cell, 1 to 27.
ALPHA	1	the blend of the particle update: 1 is FLIP, 0 is PIC.
PHYSICAL_DIFFUSION	0	1 solves the viscous term.
DIFFUSION_SOLVER	IMPLICIT	IMPLICIT (one solve) or ADI (split by direction, cheaper).
FLIP_VISCOUS_PRESSURE	0	1 puts the body force and the old pressure gradient inside the viscous solve, for very viscous liquids.
FLIP_DENSITY_CORRECTION	0	1 moves the particles toward the rest density every step, which stops the liquid's volume from drifting.
FLIP_DENSITY_SURFACE_CLAMP	1	with the correction: surface cells are not pulled inward.

SPH

Key	Default	Meaning
SPH_KERNEL_RADIUS	0.05	the kernel support radius, in metres: the method's resolution.
SPH_PARTICLE_SPACING	0	the seeding pitch, in metres; 0 means the kernel radius over 2.6.
SPH_SOUND_SPEED	20	the sound speed c_0, in m/s, at least ten times the fastest flow.
SPH_DELTA	0.1	the density diffusion coefficient.
SPH_ALPHA	0.02	the artificial viscosity.

EXPLICIT

Key	Default	Meaning
EXPLICIT_RELAXATION	1.7	the SOR factor ω, 0.5 to 2.
EXPLICIT_TOLERANCE	10^{-6}	the largest residual, relative to the largest right-hand side.
EXPLICIT_MAX_ITERATIONS	10000	the sweep cap.

10 The scene

The scene is a small language with one directive per line. A value is a number or an expression in `x y z t dh Lx Ly Lz`. In an expression a leading minus sign binds tighter than `^`, so write a Gaussian as `exp(0-(...)^2)`.

```
boundary xmin inlet u="1.5*4*y*(0.41-y)/0.41^2" v="0"
boundary xmax pressure_outlet
boundary ymin noslip
boundary ymax noslip
region solid cylinder axis=z center=(0.2,0.2) radius=0.05
scalar smoke scheme=upwind form=flux
probe p_front p at=(0.15,0.2,0)
```

Directive	Meaning
`boundary <face> <kind>`	`face` is one of `xmin` to `zmax`. `kind` is `noslip`, `slip`, `moving`, `inlet`, `pressure_outlet` or `outflow` (Section 4.7). A `moving` wall takes a tangential speed. An `inlet` takes its velocity, and `p=` sets the pressure level when the system has no other. A `pressure_outlet` takes `p=`, or 0 without it. Scalars take their values here, for example `smoke="1"`. A face that is not named is a no-slip wall.
`region <solid|fluid> <shape>`	The shapes are `box min= max=`, `sphere center= radius=`, `cylinder axis= center= radius=`, `plane point= normal=`, `mesh file= center= rotate= scale=` (a closed STL) and `naca00XX center= chord= pivot= rotate= [span=]`. `invert` keeps the outside. A `fluid` region is where a particle method seeds its liquid.
`scalar <name>`	`solver=adi|none` (default `adi`), `scheme=central|upwind`, `form=flux|advective`, `epsilon=`, `gamma=` and `source="..."`. With `solver=none` the field is held and its boundary set; `scheme` and `form` do not apply, and `epsilon`, `gamma` and `source` are kept for the method that updates it. It must come before any boundary that names it.
`force`	`u= v= w=`. A body force, as an acceleration, added to the config's `Fu`, `Fv` and `Fw`.
`initial`	`u= v= w=`. The velocity at time 0.
`probe <name> <quantity>`	`at=(x,y,z)` and an optional `radius=`. Each probe adds one telemetry column (Section 10.1).

The parser is `src/headers/Core/Scene.h`. `ConfigReader` loads the block into the global `SCENE` and binds the grid's boundary functions and cell types to it, so no solver reads the scene text. Within a list the later line wins: a boundary declared last owns the edges and corners it shares, and a `fluid` region after a `solid` one carves a hole in it.

10.1 Probes

A probe writes one value per telemetry row, in a column named after the probe and its quantity, for example `S1_p`. The value is the raw one, read from the field the step just produced. Any filtering or post-processing is done afterwards, on the CSV.

```
probe S1    p     at=(1.61,0.003,0.075) radius=0.0021   # a wall pressure sensor
probe H1    level at=(1.31,0,0.075)                     # a wave gauge
probe jet   top   at=(0.8,0,0)                          # the highest liquid, spray included
probe smoke smoke at=(0.5,0.2,0)                        # a declared scalar
```

The quantity is `p`, `u`, `v`, `w`, `speed`, `level`, `top` or the name of a declared scalar. In 2D the third coordinate may be left out.

Pressure and scalars. Bilinear (2D) or trilinear (3D) interpolation between the cell centres c around the point, with weights w_c, solid cells left out and the rest renormalised,

$$P = \frac{\sum_{c \text{ not solid}} w_c\, p_c}{\sum_{c \text{ not solid}} w_c}.$$

A probe on a wall therefore reads the fluid beside it.

Velocity. `u`, `v` and `w` are interpolated on the staggered faces, and are 0 in a solid cell. `speed` is the magnitude of the three.

Radius. With a radius R, every quantity except `level` and `top` becomes the volume-weighted average over the cells c whose centre lies within R of the probe and that are not solid,

$$P = \frac{\sum_{c \in S} V_c P_c}{\sum_{c \in S} V_c}, \qquad S = \left\{ c \ : \ |\mathbf{x}_c - \mathbf{x}_{\text{probe}}| \le R, \ c \text{ not solid} \right\},$$

which models a sensor that reports the load over its face. When no centre lies within R the probe falls back to the point value.

Level. A wave gauge: the height of the liquid column connected to the bed. Each grid column is walked up from the bottom, stepping over solid cells below the liquid, and its height h_j is the top face of the last fluid cell before the first cell that is not fluid. The probe's column is used, or with a radius every column whose centre is within R, averaged by their cross-sections A_j,

$$H = \frac{\sum_j A_j \, h_j}{\sum_j A_j}.$$

Top. The largest y among the particles within R of the probe's vertical line (half a cell when R is 0), spray included. Without particles, the top face of the highest fluid cell in the column.

```
struct Probe { std::string name, quantity; ProbeQuantity kind; double at[3]; double radius; };
```

One probe as the scene declared it. `SCENE.probes` holds them in declaration order.

```
double EvaluateProbe(MAC *grid, const ParticleSet *particles, const Probe &probe);
```

Reads one probe. The telemetry calls it on `GRID_SOL` after each step. It does not change the grid.

Parameters

- `grid` – the grid to read.
- `particles` – the particle set, for `top`. It may be empty.
- `probe` – the probe.

Returns the probe's value, in the units of its quantity.

Example

```
for (const Probe &pr : SCENE.probes)
    std::printf("%s_%s = %g\n", pr.name.c_str(), pr.quantity.c_str(),
                EvaluateProbe(SIMULATION.GRID_SOL, &SIMULATION.particles, pr));
```

11 Telemetry

With `EXPORT_ENABLED = 1` the run writes `telemetry.csv` in its output folder: a header, then one row every `EXPORT_TELEMETRY_INTERVAL` steps and one at the end of the run. Every value is read from the field the step has just produced. A group of columns is left out when its `EXPORT_COLUMN_*` flag is 0 or, for the last three groups, when its export flag is off. The columns appear in the order of the table.

Column	Flag	Value
`iteration`	always	the number of steps completed.
`time`	`_TIME`	the simulated time of the field, in seconds.
`div_sum`	`_DIVERGENCE`	$\sum_c \lvert(\nabla \cdot \mathbf{u})_c\rvert$ over the cells inside the domain, in 1/s; how well the projection made the field divergence free.
`advcfl`, `diffcfl`, `cell_reynolds`	`_CFL`	$\Delta t\,\lvert u\rvert_{\max}/h$ and $\nu\,\Delta t/h^2$ with h the smallest cell; the largest $\lvert u_d\rvert\,h_d/\nu$ over the faces, h_d the spacing along the face's own axis.
`residual`	`_RESIDUAL`	$\max\lvert\mathbf{u}^{n+1}-\mathbf{u}^n\rvert$ over every face: how far the flow still moves per step. `STOP_RESIDUAL` tests it.
`<step>_timing`	`_STEP_TIMINGS`	one column per registered step: its wall time, in seconds.
`<step>_counter`	`_STEP_TIMINGS`	one column per step: its work count, -1 for a step that reports none. `pressure_counter` is the pressure solve's iterations.
`kinetic_energy`, `enstrophy`, `dE_dt`	`_FLOW_INTEGRALS`	volume averages per unit mass over the fluid: $E = \frac{1}{V}\int \frac{1}{2}\lvert\mathbf{u}\rvert^2\,dV$, $Z = \frac{1}{V}\int \frac{1}{2}\lvert\boldsymbol{\omega}\rvert^2\,dV$, and the backward difference of E between rows (`nan` on the first).
`cl`, `cd`, `cd_cv`, `cl_cv`, `ld_ratio`	`EXPORT_AERODYNAMICS` with `_CL`, `_CD`, `_LD_RATIO`	the force coefficients of the solids inside the domain (below).
`FluidHeight`, `FluidFront`, `FluidVolume`	`EXPORT_DAMBREAK` with `_FLUID_HEIGHT`, `_FLUID_FRONT`	the free surface (below).
`<name>_<quantity>`	always	one column per probe of the scene (Section 10.1).

Force coefficients. F_x (drag) and F_y (lift) are the force of the fluid on every solid inside the domain, per unit span in 2D. The coefficients are

$$C_d = \frac{2F_x}{\rho U^2 S}, \qquad C_l = \frac{2F_y}{\rho U^2 S}, \qquad S = \begin{cases} L & \text{2D,} \\ L\,L_2 & \text{3D,} \end{cases}$$

with U = `MEAN_VELOCITY`, L = `CHARACTERISTIC_LENGTH` and L_2 = `CHARACTERISTIC_LENGTH_2`. `cd` and `cl` integrate the stress over the staircase faces of the solid. `cd_cv` and `cl_cv` take the momentum balance over a box around the body,

$$\mathbf{F} = \oint_{\partial B} \left(\boldsymbol{\sigma}\cdot\mathbf{n} - \rho\,\mathbf{u}\,(\mathbf{u}\cdot\mathbf{n})\right) dS - \frac{d}{dt}\int_B \rho\,\mathbf{u}\,dV,$$

with the box half the body's size beyond it on every side. The balance never touches the staircase, so it converges where the surface integral does not; prefer it on a curved body. `ld_ratio` is C_l/C_d.

Free surface. On the cell types of the field (in 3D, on the plane through the middle of z): `FluidHeight` is the top face of the highest fluid cell, and `FluidFront` the right face of the fluid cell furthest along x, both in metres. `FluidVolume` is the number of fluid cells times their volume (an area in 2D); it is written for a particle method with a grid, where it shows the liquid's volume drift.

12 The user interface

`src/headers/Core/SimulationManagerUI.h` declares four functions. `src/UI/SimulationManagerUI.cpp` defines them, and only the GUI program compiles it. The setup screen, the viewer and the window sit behind these four functions. `main` calls them from its own loop. The manager draws nothing and knows nothing about them.

```
bool InitializeUI(std::string &configFile);
```

Opens the window and runs the setup screen until the user presses Start or closes the window.

Parameters

- `configFile` – set to the path of the config the user chose. It is the file the setup screen saved last.

Returns true when the user pressed Start, false when the window was closed.

```
bool UpdateUI();
```

Draws one frame of the viewer and handles the window's controls, which run, pause, step and restart the simulation. Call it once after each `StepSimulation`.

Returns false once the window is closed, or when Restart was pressed.

```
bool RestartUI(std::string &configFile);
```

After `UpdateUI` returned false and the run was disposed: when that was a restart, runs the setup screen again.

Parameters

- `configFile` – set to the path of the config the user chose.

Returns true when the user pressed Start, false when the window is closing.

```
void DisposeUI();
```

Closes the window and releases the graphical libraries. It is safe to call after `InitializeUI` returned false.

Example

```
std::string config;
if (!InitializeUI(config)) { DisposeUI(); return 0; }
do
{
    SimulationManager::InitializeSimulation(config);
    Exports::InitializeTelemetry();
    do SimulationManager::StepSimulation(); while (UpdateUI());
    SimulationManager::DisposeSimulation();
} while (RestartUI(config));
DisposeUI();
```